CS24307 • COMPLETE MASTER GUIDE

Artificial Intelligence (CS24307) Full Course Master

Exhaustive 38–Topic Textbook, Python Lab Guide & Solved University Question Bank

MODULE 1 OF 5

Module 1: Intelligent Agents & PEAS Framework

Topics 1 to 7 • Foundations, Evolution, Rationality & 5 Agent Types

Module 1 Table of Contents (Topics 1 to 7)

Topic 1: Foundational Definitions & The Four AI Perspectives

Artificial Intelligence (AI) is the science and engineering of synthesizing computational artifacts capable of performing cognitive tasks traditionally associated with human intellect: perception, inductive/deductive reasoning, learning from experiential data, goal-directed planning, natural language comprehension, and autonomous decision-making under uncertainty.

Dimension	Human-Centric Dimension (Empirical / Cognitive)	Rationality-Centric Dimension (Ideal / Normative)
Reasoning (Thought)	**Thinking Humanly:** Cognitive Science approach. Formulates computational models of human mental processes via fMRI neuroscience and introspection (e.g. Newell & Simon's General Problem Solver).	**Thinking Rationally:** Laws of Thought approach. Formalizes reasoning using strict deductive logic and syllogisms ($\forall x(P(x) \rightarrow Q(x))$) originating with Aristotle.
Behavior (Action)	**Acting Humanly:** The Turing Test approach (Alan Turing, 1950). An entity is intelligent if an interrogator cannot distinguish its conversational responses from a human.	**Acting Rationally (Modern Standard):** Rational Agent approach (Russell & Norvig). An agent operates to maximize its *expected performance measure* given its percept sequence and built-in knowledge.

Deep Dive: The 6 Sub-disciplines Required for the Total Turing Test

1. **Natural Language Processing (NLP):** Comprehending, parsing, and generating human spoken/written communication.
2. **Knowledge Representation (KR):** Storing structured ontological assertions, episodic memories, and causal rules.
3. **Automated Reasoning:** Drawing logically sound inferences, answering queries, and resolving contradictions.
4. **Machine Learning (ML):** Adapting internal statistical weights to extrapolate patterns and survive novel scenarios.
5. **Computer Vision (Total Turing Test):** Perceiving and segmenting 3D physical objects from visual sensor streams.
6. **Robotics & Actuation (Total Turing Test):** Manipulating physical objects and navigating complex terrain.

Topic 2: Historical Evolution, Dartmouth 1956 & AI Winters

Figure 1.1: Chronological Evolution of Artificial Intelligence from Symbolic Logic to Deep Generative Models

Topic 3 & 4: Intelligent Agents & Formal Concept of Rationality

An **Agent** is anything that perceives its environment through physical or software sensors and acts upon that environment via actuators. Formally, an agent is mathematically characterized by an **Agent Function** f:

The Mathematical Agent Function:

$$\mathbf{f} : \mathcal{P}^* \longrightarrow \mathcal{A}$$

Where $\mathcal{P}^*$ represents the history of all percept sequences perceived up to the current discrete timestep t, and $\mathcal{A}$ is the set of executable actions. The concrete implementation of f running on physical compute hardware is the **Agent Program**.

⚖ Rationality vs. Omniscience

- **Rationality:** Maximizes *expected performance* given the historical percept sequence $\mathcal{P}^*$ and prior built-in knowledge. It does NOT require perfection.
- **Omniscience:** Knows the *actual outcome* of its actions in advance (impossible in non-deterministic or partially observable real-world environments).

Topic 5: PEAS Specification & Environmental Taxonomy

To engineer an intelligent system, one must formally define its **PEAS Framework**:

System Domain	Performance Measure (P)	Environment (E)	Actuators (A)	Sensors (S)
Autonomous Taxi	Safety, speed, legal compliance, passenger comfort, fuel economy, profit.	Public roads, pedestrians, traffic lights, weather conditions.	Steering wheel, accelerator, brakes, horn, turn signals.	LiDAR, RADAR, cameras, GPS, odometer, engine sensors.
Medical Diagnosis	Patient health recovery, diagnosis accuracy, minimized costs and side effects.	Hospital ward, patient symptoms, lab test facilities.	Treatment plan, prescription dosage, surgery referral.	Blood pressure monitor, EHR records, lab results.
Chess AI (Stockfish)	Winning game ($+1$), draw (0), checkmating opponent, piece advantage.	8×8 Chess board, opponent player, clock timer.	Move piece command ($e2 \rightarrow e4$).	Board state matrix representation.
Automated Warehouse Robot	Items picked/hour, zero item drops, collision-free transit, battery life.	Warehouse aisles, shelving racks, human workers.	Differential drive wheels, robotic gripping arm.	Ultrasonic rangefinders, barcode scanners, depth camera.

Topic 6: The 5 Classical Agent Structural Architectures

Agent Architecture	Internal Decision Logic	Pros & Limitations
1. Simple Reflex Agent	Selects actions strictly based on the *current instantaneous percept* using condition-action rules (`IF car-in-front-braking THEN apply-brakes`). Ignores history.	Extremely fast; fails completely in partially observable environments (falls into infinite loops).
2. Model-Based Reflex Agent	Maintains an internal **State** tracking how the unobserved world evolves independently and how the agent's own actions affect the world.	Can handle partial observability; requires accurate world physics modeling.
3. Goal-Based Agent	Combines internal state with explicit **Goal Descriptions** to evaluate whether candidate action sequences achieve desired end states (uses Search & Planning).	Highly flexible (goals can change dynamically); slower than reflex lookup.
4. Utility-Based Agent	Maps states to real-valued scalar **Utility Values** $U(s) \in \mathbb{R}$ to make optimal trade-offs between competing goals and risk profiles under uncertainty.	Provides mathematically optimal decision making; computationally intensive.
5. Learning Agent	Divided into 4 modules: *Learning Element* (improves performance), *Critic* (evaluates against standard), *Performance Element* (acts), and *Problem Generator* (experiments).	Can operate in initially unknown environments and achieve superhuman competence over time.

Topic 7: Master University Examination Solved Question Bank (10 Solved Questions)

Q1. Prove why a Simple Reflex Agent cannot operate rationally in a partially observable environment like the Vacuum World. (8 Marks)

In a 2-room vacuum cleaner world where the agent can sense only its current location and cleanliness (not whether the other room is dirty), if both rooms are clean and the agent has no memory, its rule `IF clean THEN MoveRight` followed in Room B by `IF clean THEN MoveLeft` produces an **infinite oscillating loop**. Without internal state memory, it cannot deduce that both rooms have already been cleaned!

Q2. Formally classify the Environment of: (a) Poker, (b) Autonomous Driving, and (c) Crossword Puzzle across all 6 environmental dimensions. (10 Marks)

- **(a) Poker:** Partially Observable (hidden cards), Stochastic (card shuffling), Sequential (bets affect later rounds), Static (rules/pot do not change while thinking), Discrete (finite chips/cards), Multi-Agent (Competitive/Adversarial).
- **(b) Autonomous Driving:** Partially Observable (blind spots, occluded vehicles), Stochastic (erratic drivers, tire slip), Sequential, Dynamic (pedestrians and traffic move while vehicle decides), Continuous (steering angle, velocity), Multi-Agent (Cooperative & Competitive).
- **(c) Crossword Puzzle:** Fully Observable (entire grid visible), Deterministic (words do not change), Sequential, Static, Discrete (letters and grid cells), Single-Agent.

Q3. Detail the components of a Learning Agent with an architectural block diagram. (8 Marks)

A Learning Agent decomposes into four distinct structural units:
1. **Learning Element:** Responsible for making improvements by analyzing feedback from the critic.
2. **Critic:** Evaluates the agent's behavior against an external performance standard (provides reward/penalty signals).
3. **Performance Element:** The operational agent that selects external actions based on percepts (e.g. reflex, model, goal, or utility core).
4. **Problem Generator:** Suggests novel exploratory actions leading to new experiences rather than just exploiting known sub-optimal paths.

Q4. What is the Chinese Room Argument by John Searle? How does it challenge Strong AI? (8 Marks)

John Searle's Chinese Room Thought Experiment (1980): A monolingual English speaker sits in a closed room with a rulebook translating incoming Chinese characters into corresponding Chinese output responses. To an outside observer, the room passes the Turing Test in Chinese. However, the human inside does not understand a single word of Chinese; they are merely performing *syntactic symbol manipulation* without *semantic intentionality or true understanding*. Searle concludes that functional execution of a computer program cannot produce genuine human-like consciousness (Strong AI).

Q5. Explain the Subsumption Architecture proposed by Rodney Brooks. How does it contrast with Classical Symbolic AI? (8 Marks)

• **Classical Symbolic AI:** Relies on a centralized "Sense-Plan-Act" pipeline with explicit symbolic internal world models.
• **Subsumption Architecture (Behavior-Based AI):** Eliminates central representations ("The world is its own best model"). The system is built from parallel, asynchronous, reactive behavioral layers (e.g. Layer 0: Avoid Obstacles; Layer 1: Wander; Layer 2: Explore). Higher layers can *suppress* inputs or *subsume* outputs of lower layers, achieving robust physical reactivity in dynamic environments without computationally prohibitive planning bottlenecks!

Q6. Compare Goal-Based Agents and Utility-Based Agents with decision-theoretic examples. (8 Marks)

• **Goal-Based Agents:** Seek binary achievement of defined goals ($G(s) \in \{\text{True}, \text{False}\}$). They lack the ability to trade off conflicting criteria (e.g. speed vs safety) or quantify partial goal satisfaction.
• **Utility-Based Agents:** Map states to a real-valued scalar utility function $U : \mathcal{S} \to \mathbb{R}$. When multiple goals conflict (e.g., getting to the airport quickly vs spending minimal fuel vs avoiding toll roads), a utility agent computes the expected utility $\mathbb{E}[U] = \sum_s P(s \mid a)U(s)$ and selects the action that maximizes expected utility (Maximum Expected Utility Principle).

Q7. Differentiate between Epistemic Actions and Pragmatic Actions with robotic exploration examples. (6 Marks)

• **Pragmatic Actions:** Actions intended to change the physical world state to bring the agent closer to its goal (e.g., vacuum cleaner picking up dirt, autonomous car accelerating).
• **Epistemic Actions:** Actions whose primary purpose is to acquire information and reduce uncertainty in a partially observable world (e.g., exploring an unmapped room, running a medical diagnostic blood test, looking around a blind corner before turning).

Q8. What are the key limitations of Expert Systems that led to the Second AI Winter? (8 Marks)

1. **Knowledge Acquisition Bottleneck:** Hand-crafting thousands of IF-THEN rules from human experts was expensive and slow.
2. **Brittleness:** Expert systems could not handle edge cases outside their narrow rule sets.
3. **Lack of Commonsense Reasoning:** Inability to make simple real-world deductions.
4. **High Maintenance Costs:** Conflicting rules caused unpredictable system behavior as rule bases grew.

Q9. Explain the concept of Bounded Rationality proposed by Herbert Simon. (6 Marks)

Perfect rationality assumes unbounded computation to find optimal actions. In the real world, agents face strict computational, memory, and time constraints. **Bounded Rationality** dictates that an agent acts as rationally as possible given its finite computational resources (satisficing rather than global optimization).

Q10. Formulate the State-Space representation of the Water Jug Problem (4-Gallon and 3-Gallon Jugs to measure 2 Gallons). (8 Marks)

• **State Space:** Ordered pair (x, y) where $x \in \{0, 1, 2, 3, 4\}$ (water in 4-gal jug) and $y \in \{0, 1, 2, 3\}$ (water in 3-gal jug).
• **Initial State:** $(0, 0)$.
• **Goal State:** $(2, y)$ for any y.
• **Production Rules / Operators:**
1. Fill 4-gal jug: $(x, y) \to (4, y)$ if $x < 4$.
2. Fill 3-gal jug: $(x, y) \to (x, 3)$ if $y < 3$.
3. Empty 4-gal jug: $(x, y) \to (0, y)$ if $x > 0$.
4. Empty 3-gal jug: $(x, y) \to (x, 0)$ if $y > 0$.
5. Pour from 3 to 4: $(x, y) \to (\min(4, x + y), y - (\min(4, x + y) - x))$.
6. Pour from 4 to 3: $(x, y) \to (x - (\min(3, x + y) - y), \min(3, x + y))$.
• **Solution Trace:** $(0, 0) \to (0, 3) \to (3, 0) \to (3, 3) \to (4, 2) \to (0, 2) \to (2, 0)$ (Goal reached!).

Topic 7.5: Philosophical Foundations & Minds vs. Machines

The philosophical foundations of artificial intelligence examine the feasibility, limits, and metaphysical nature of synthetic cognition.

Philosophical Position	Core Claim	Key Objections & Philosophical Refutations
Strong AI (Functionalism)	An appropriately programmed computer with the right inputs and outputs literally *has a mind* and possesses genuine consciousness and understanding.	Searle's Chinese Room argument: Syntax alone cannot generate intentionality or semantics ($S \not\Longrightarrow \Sigma$).
Weak AI (Instrumentalism)	Computers can only simulate human cognition and act *as if* they were intelligent, providing powerful analytical tools without actual subjective experience.	Accepted by mainstream engineering; focuses on operational competence and mathematical performance measures.
Gödel's Incompleteness Argument (Lucas / Penrose)	By Gödel's First Incompleteness Theorem, any consistent formal axiomatic system contains true mathematical statements that cannot be proven within the system. Human mathematicians can perceive these truths; hence human minds are non-algorithmic.	Computers, like humans, are not infallible; Gödel's theorem applies only to consistent, closed formal systems, whereas humans learn heuristically from experience.
Dreyfus' Critique of Symbolic AI	Human expertise relies on tacit, bodily intuitive know-how and situational context (phenomenology) rather than explicit symbolic rules and propositional logic.	Led to modern embodied robotics, connectionism, and deep reinforcement learning from sensorimotor interactions.

🏛️ Step-by-Step Analysis: Alan Turing's 1950 Objections and Responses

In his landmark 1950 paper *"Computing Machinery and Intelligence"*, Alan Turing anticipated and systematically refuted nine major objections to machine intelligence:

1. **The Theological Objection:** "Thinking is a function of man's immortal soul; God gave an immortal soul to every man and woman, but not to any other animal or machine."
   Turing's Refutation: This restricts the omnipotence of God; why couldn't God grant a soul to a computational machine if He wished?
2. **The 'Heads in the Sand' Objection:** "The consequences of machines thinking would be too dreadful. Let us hope and believe that they cannot do so."
   Turing's Refutation: An emotional fear of losing human intellectual superiority rather than a rational scientific argument.
3. **The Mathematical Objection (Gödel):** Finite machines are subject to Gödelian limits on provability.
   Turing's Refutation: Humans frequently make errors, suffer from memory bounds, and fail to prove mathematical truths; machines need not be infallible gods to match human intellect.
4. **The Argument from Consciousness:** "Not until a machine can write a sonnet or compose a concerto because of thoughts and emotions felt, and not by the chance fall of symbols, could we agree that machine equals brain." (Jefferson).
   Turing's Refutation: The only way to know what a person feels is to *be* that person (Solipsism). In ordinary life, we accept behavioral communication as evidence of other minds.
5. **Arguments from Various Disabilities:** "A machine can do many things, but it will never be able to: be kind, have a sense of humor, fall in love, make mistakes, learn from experience, enjoy strawberries and cream."
   Turing's Refutation: These limitations are artifacts of current narrow engineering rather than fundamental physical laws of computation.
6. **Lady Lovelace's Objection (1842):** "The Analytical Engine has no pretensions to originate anything. It can do whatever we know how to order it to perform."
   Turing's Refutation: Machines can surprise humans by discovering unexpected proofs and emergence from complex non-linear interactions. Machine learning allows systems to acquire rules never explicitly programmed.
7. **Argument from Continuity in the Nervous System:** The biological brain is a continuous neural medium, whereas digital computers are discrete-state machines.
   Turing's Refutation: A discrete-state machine can approximate any continuous differential equation system to arbitrary mathematical precision.
8. **The Argument from Informality of Behavior:** "It is not possible to produce a set of rules purporting to describe what a man should do in every conceivable set of circumstances."
   Turing's Refutation: Humans follow probabilistic, statistical laws of behavior rather than rigid deterministic condition-action rule tables.
9. **The Argument from Extrasensory Perception (ESP):** Telepathy and telekinesis.
   Turing's Refutation: If telepathy exists, test conditions can be engineered in Faraday cages.

🏛️ **Complete Mathematical Formalism: The Agent-Environment Interaction Cycle**

Let discrete timesteps be $t \in \{0, 1, 2, \dots\}$. The interaction between agent and environment proceeds as:

$$\mathbf{e_t} \in \mathcal{E} \xrightarrow{\text{Sensor Matrix } \mathbf{S}} \mathbf{p_t} \in \mathcal{P} \xrightarrow{\text{History Sequence } \mathbf{h_t}=(\mathbf{p_0},\dots,\mathbf{p_t})} \mathbf{a_t} = \mathbf{f}(\mathbf{h_t}) \in \mathcal{A} \xrightarrow{\text{Transition Dynamics } \mathcal{T}} \mathbf{e_{t+1}}$$

The total performance score over horizon T is given by an evaluation functional:

$$\mathbf{V}(\mathbf{f}) = \mathbb{E}_{\mathcal{T}} \left[\sum_{\mathbf{t=0}}^{\mathbf{T}} \gamma^{\mathbf{t}} \mathbf{R}(\mathbf{e_t}, \mathbf{a_t}, \mathbf{e_{t+1}}) \right]$$

A globally rational agent function $\mathbf{f}^*$ maximizes expected cumulative discounted reward:

$$\mathbf{f}^* = \arg\max_{\mathbf{f}} \mathbf{V}(\mathbf{f}) = \arg\max_{\mathbf{f}} \mathbb{E} \left[\sum_{\mathbf{t=0}}^{\infty} \gamma^{\mathbf{t}} \mathbf{R_t} \right]$$

Q13. Compare the Symbolic Paradigm (Good Old-Fashioned AI / GOFAI) with the Connectionist Paradigm. (10 Marks)

Dimension	Symbolic Paradigm (GOFAI)	Connectionist Paradigm (Neural / Sub-symbolic)
Representation	Explicit, discrete symbols, predicates, logic rules, semantic ontologies.	Distributed continuous vector embeddings, synaptic weight matrices.
Inference Mechanism	Deductive logic, theorem proving, search algorithms, unification.	Non-linear activation propagation, matrix multiplication, backpropagation gradient descent.
Noise Tolerance	Brittle; single missing rule or syntactic typo causes total failure.	Graceful degradation; robust to noisy sensors and missing features.
Interpretability	Transparent white-box reasoning; generates step-by-step resolution proofs.	Black-box models; high dimensional weights difficult for humans to audit.
Strengths	Exact mathematical reasoning, planning, rule compliance, explanation.	Perception (Vision, Audio), continuous control, pattern recognition.

Q14. Explain the Concept of an Architecture in Russell & Norvig's formulation: $\text{Agent} = \text{Architecture} + \text{Program}$. (6 Marks)

The **Architecture** is the physical computational device (CPU, GPU, FPGA, micro-controller) along with hardware sensors (cameras, LiDAR, encoders) and actuators (motors, solenoids, speakers). The **Agent Program** is the concrete algorithm or software function that runs on the architecture, taking incoming sensor raw feeds and computing output actuator voltages. The architecture must provide sufficient compute throughput and low sensor latency to allow the agent program to make decisions within real-time deadlines!

Q15. Detail the 4 Classical Types of Environment Observability and give real-world examples. (8 Marks)

1. **Fully Observable:** The agent's sensors give it access to the complete state of the environment at each point in time (e.g., Chess, Go, 8-Puzzle).
2. **Partially Observable:** Sensor noise, occlusions, or hidden opponent state prevent complete knowledge (e.g., Autonomous driving with fog/blind spots, Poker with face-down cards).
3. **Unobservable (Sensorless / Conformant):** The agent has NO sensors at all. It must find a sequence of actions that achieves the goal regardless of the initial starting state (e.g., Automated vacuum cleaner running in a completely dark room finding clean states via blind sweeping).
4. **Semi-Observable:** Only high-level aggregated signals are available (e.g., Macro-economic central banking monitoring quarterly inflation indicators).

Topic 7.6: Detailed Mathematical Properties of State Space & Perception

Formal Agent Perception History & Action Space:

$$\mathcal{P} = \{\mathbf{p_1}, \mathbf{p_2}, \dots, \mathbf{p_k}\} \implies \mathcal{P}^* = \bigcup_{\mathbf{t=0}}^{\infty} \mathcal{P}^{\mathbf{t}}$$

$$\text{Number of Possible Agent Functions over Horizon } \mathbf{T}: \ |\mathcal{A}|^{|\mathcal{P}|^{\mathbf{T}}}$$

Combinatorial Significance: For even modest $|\mathcal{P}| = 10$ and $|\mathcal{A}| = 5$ over $T = 10$ steps, there are $5^{10^{10}}$ candidate agent functions! This astronomical search space proves why tabular lookup reflex agents are physically impossible and why compact, generalizing representations (logic, heuristics, neural networks) are mandatory.

🏛️ **Step-by-Step Solved Problem: Agent Performance Metric Engineering**

An autonomous street cleaning robot is deployed in an urban city center. Compare two proposed performance measures:

- **Metric 1:** Amount of dirt collected in the robot's onboard disposal bin over 24 hours.
- **Metric 2:** Cleanliness of the entire street network averaged over 24 hours.

Critical Agent Analysis:

- Under **Metric 1**, a rational agent maximizes dirt in its bin. To maximize this metric, the agent might clean up a pile of dirt, dump it back onto the clean pavement, and sweep it up again repeatedly!
- Under **Metric 2**, the agent is rewarded strictly for the *desired state of the external environment* (clean streets), preventing perverse reward hacking!
- **Core Principle (Russell & Norvig):** Design performance measures according to what you want to achieve in the environment, NOT according to how you think the agent should behave!

Q16. Explain the Concept of Software Agents (Softbots) and Web Crawling Agents. (8 Marks)

A **Softbot** is an intelligent agent whose environment is entirely digital (operating systems, web networks, databases, cloud infrastructure) rather than physical hardware. Example (Search Engine Indexing Spider):
• **P:** Freshness of search index, crawl coverage, low server load, high page pagerank relevance.
• **E:** The World Wide Web (HTTP/HTTPS protocols, HTML/DOM trees, sitemaps, robots.txt constraints).
• **A:** HTTP GET requests, URL extraction, indexing pipeline dispatch, rate-limiting sleep calls.
• **S:** HTTP status codes, HTML content strings, header response times.

Topic 7.7: Ethical Foundations, Safety & Fairness in Autonomous Systems

As autonomous agents transition from laboratory benchmarks to real-world deployment, ethical constraints and safety bounds must be formalized as mathematical invariants within utility functions.

Ethical Dimension	Theoretical Formalization	Industrial Deployment Standard
Value Alignment (Nick Bostrom)	Ensuring an agent's objective utility function $\mathcal{U}$ strictly aligns with true human intent rather than literal narrow specifications (preventing the Paperclip Maximizer catastrophe).	Inverse Reinforcement Learning (IRL), Cooperative Inverse Reinforcement Learning (CIRL).
Algorithmic Fairness	**Demographic Parity:** $P(\hat{Y} = 1 \mid A = 0) = P(\hat{Y} = 1 \mid A = 1)$; **Equalized Odds:** True positive and false positive rates are equal across protected demographic attributes A.	AI-driven hiring tools, automated credit risk scoring, judicial recidivism prediction.
Explainability & Interpretability (XAI)	Generating post-hoc explanations for black-box neural decisions via Local Interpretable Model-agnostic Explanations (LIME) and SHAP (Shapley Additive exPlanations).	EU AI Act high-risk compliance, FDA medical device diagnostic validation.

🏛️ **Step-by-Step Solved Problem: Formal Mathematical Fair Decision Metric**

A credit underwriting agent evaluates loan applications from two demographic groups ($A = 0, A = 1$):

- Group $A = 0$: 1000 applicants, 200 approved ($\hat{Y} = 1$), 150 repaid ($Y = 1$).
- Group $A = 1$: 2000 applicants, 600 approved ($\hat{Y} = 1$), 450 repaid ($Y = 1$).

1. Demographic Parity Check:

$$P(\hat{Y} = 1 \mid A = 0) = \frac{200}{1000} = 0.20 \qquad P(\hat{Y} = 1 \mid A = 1) = \frac{600}{2000} = 0.30$$

$$\text{Disparate Impact Ratio: } \mathbf{\frac{0.20}{0.30}} = \mathbf{0.667} < \mathbf{0.80} \implies \text{Violates } 80\% \text{ EEOC Rule!}$$

2. True Positive Rate (Equality of Opportunity):

$$\text{TPR}_{A=0} = \frac{\text{True Positives}}{\text{Actual Positives}} = \frac{150}{200} = 0.75 \qquad \text{TPR}_{A=1} = \frac{450}{600} = 0.75$$

$$\text{Equality of Opportunity is SATISFIED } (\mathbf{\text{TPR}_0 = \text{TPR}_1 = 0.75})!$$

MODULE 2 OF 5

Module 2: Search Algorithms & Game Playing

Topics 8 to 13 • BFS/DFS/IDDFS, A* Admissibility Proofs & Alpha-Beta Pruning

Module 2 Table of Contents (Topics 8 to 13)

Topic 8: Formal Problem Formulation & State Space Representation

A classical search problem is formally defined by a 5-tuple:

$$\mathcal{P} = \langle \mathbf{S_0}, \text{Actions}(\mathbf{s}), \text{Result}(\mathbf{s}, \mathbf{a}), \text{GoalTest}(\mathbf{s}), \mathbf{c}(\mathbf{s}, \mathbf{a}, \mathbf{s'}) \rangle$$

1. **Initial State (S_0):** The starting state of the agent.
2. **Actions ($Actions(s)$):** The set of legal actions applicable in state s.
3. **Transition Model ($Result(s, a)$):** Returns the successor state s' produced by action a in state s.
4. **Goal Test ($GoalTest(s)$):** Boolean function determining if state s is a goal state.
5. **Path Cost ($c(s, a, s')$):** Step cost of taking action a from state s to s'. Total path cost is $g(n) = \sum \text{step costs}$.

Step-by-Step Problem Formulation: The 8-Puzzle Problem

- **State:** 3×3 grid configuration containing numbers $\{1, 2, \ldots, 8\}$ and one blank space ($\text{Total States} = 9!/2 = 181,440$).
- **Initial State:** Any given permutation of the 8 tiles and blank.
- **Actions:** Move Blank $\{\text{Left}, \text{Right}, \text{Up}, \text{Down}\}$.
- **Goal Test:** Matches goal matrix `[[1,2,3],[8,blank,4],[7,6,5]]` (or standard sequential `[[1,2,3],[4,5,6],[7,8,blank]]`).
- **Path Cost:** Each tile move costs 1 unit ($g(n) = \text{number of moves}$).

Topic 9: Exhaustive Uninformed (Blind) Search Algorithms

Algorithm	Data Structure	Time Complexity	Space Complexity	Complete?	Optimal?
Breadth-First Search (BFS)	FIFO Queue	$O(b^d)$	$O(b^d)$ (Severe memory bottle-neck)	**Yes** (if $b < \infty$)	**Yes** (if step costs equal)
Uniform-Cost Search (UCS)	Priority Queue by $g(n)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	**Yes** (if step cost $\geq \epsilon > 0$)	**Yes** (Cost optimal)
Depth-First Search (DFS)	LIFO Stack	$O(b^m)$ ($m = \max \text{depth}$)	$O(b \cdot m)$ (Linear memory!)	No (Infinite trees)	No
Depth-Limited Search (DLS)	LIFO Stack with limit l	$O(b^l)$	$O(b \cdot l)$	No (if $l < d$)	No
Iterative Deepening DFS (IDDFS)	LIFO Stack with increasing l	$O(b^d)$	$\mathbf{O(b \cdot d)}$ (Best of both worlds!)	**Yes**	**Yes** (if step costs equal)
Bidirectional Search	Two simultaneous Queues	$O(b^{d/2})$	$O(b^{d/2})$	**Yes**	**Yes** (if step costs equal)

Topic 10 & 11: Informed Heuristic Search (A^*) & Mathematical Proofs

A^* Search evaluates frontier nodes by the total estimated path cost function:

$$\mathbf{f(n) = g(n) + h(n)}$$

Where $g(n)$ is the exact cost from initial state to node n, and $h(n)$ is the estimated cost from node n to the closest goal state.

1. Admissibility Property (Tree-Search Optimality): A heuristic $h(n)$ is **admissible** if it never overestimates the true minimal cost $h^*(n)$ to reach the goal:

$$\mathbf{0 \leq h(n) \leq h^*(n) \quad \forall n}$$

2. Consistency / Monotonicity Property (Graph-Search Optimality): A heuristic $h(n)$ is **consistent** if for every node n and every successor n' generated by action a:

$$\mathbf{h(n) \leq c(n, a, n') + h(n')}$$

$$\text{Triangle Inequality: } \mathbf{f(n') = g(n') + h(n') = g(n) + c(n, a, n') + h(n') \geq g(n) + h(n) = f(n)}$$

Crucial Consequence: Monotonicity guarantees that $f(n)$ is non-decreasing along any path, ensuring that the first time a node is expanded in Graph-Search, the path is globally optimal!

🏛️ **Step-by-Step Solved Problem: Trace of A^* Search on Romania Map**

Find shortest path from **Arad to Bucharest** given straight-line distance heuristics h_{SLD}:

- $h(\text{Arad}) = 366, h(\text{Zerind}) = 374, h(\text{Timisoara}) = 329, h(\text{Sibiu}) = 253, h(\text{Fagaras}) = 176, h(\text{Rimnicu}) = 193, h(\text{Pitesti}) = 100, h(\text{Bucharest}) = 0$.

A^* Priority Queue Execution Trace:

1. Expand **Arad**: Front = $\{\text{Sibiu: } g = 140, f = 140 + 253 = 393;\ \text{Timisoara: } g = 118, f = 118 + 329 = 447;\ \text{Zerind: } g = 75, f = 75 + 374 = 449\}$.
2. Select lowest f: **Sibiu ($f = 393$)** $\rightarrow$ Expand Sibiu: Successors: $\{\text{Fagaras: } g = 140 + 99 = 239, f = 239 + 176 = 415;\ \text{Rimnicu: } g = 140 + 80 = 220, f = 220 + 193 = 413\}$.
3. Select lowest f: **Rimnicu Vilcea ($f = 413$)** $\rightarrow$ Expand Rimnicu: Successors: $\{\text{Pitesti: } g = 220 + 97 = 317, f = 317 + 100 = 417;\ \text{Craiova: } g = 220 + 146 = 366, f = 366 + 160 = 526\}$.
4. Select lowest f: **Fagaras ($f = 415$)** $\rightarrow$ Expand Fagaras: Successor: $\{\text{Bucharest: } g = 239 + 211 = 450, f = 450 + 0 = 450\}$.
5. Select lowest f: **Pitesti ($f = 417$)** $\rightarrow$ Expand Pitesti: Successor: $\{\text{Bucharest: } g = 317 + 101 = \mathbf{418}, f = 418 + 0 = \mathbf{418}\}$.
6. Select lowest f: $\text{Bucharest } (\mathbf{f = 418}) \rightarrow$ Goal reached!

$$\text{Optimal Path: Arad} \rightarrow \text{Sibiu} \rightarrow \text{Rimnicu Vilcea} \rightarrow \text{Pitesti} \rightarrow \text{Bucharest} \quad (\text{Total Cost } = \mathbf{418})$$

Topic 12 & 13: Adversarial Search, Minimax & Alpha-Beta Pruning

In a zero-sum two-player game, $\mathbf{MAX}$ seeks to maximize the utility score while $\mathbf{MIN}$ seeks to minimize it:

$$\text{Minimax}(\mathbf{s}) = \begin{cases} \text{Utility}(s) & \text{if Terminal}(s) \\ \max_{a \in Actions(s)} \text{Minimax}(\text{Result}(s,a)) & \text{if Player}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{Minimax}(\text{Result}(s,a)) & \text{if Player}(s) = \text{MIN} \end{cases}$$

The Alpha-Beta Pruning Invariant: - α: The highest-value choice found so far at any choice point along the path for MAX (initialized to $-\infty$). - β: The lowest-value choice found so far at any choice point along the path for MIN (initialized to $+\infty$).

$$\text{Pruning Rule: Prune remaining subtree whenever } \alpha \geq \beta$$

🏛️ **Step-by-Step Solved Problem: Alpha-Beta Pruning Tree Execution**

Consider a 2-ply game tree with root MAX, children B, C, D (MIN), each having 2 terminal leaves:

- Node B: Leaves are $[3, 5] \implies \text{MIN}$ evaluates 3, updates $\beta = 3$. Evaluates 5 $\implies$ Node $B = 3$. Root MAX updates $\alpha = 3$.
- Node C: Left leaf is 2. MIN updates $\beta = 2$. Now $\alpha = \mathbf{3} \geq \beta = \mathbf{2} \implies \text{PRUNE RIGHT CHILD OF } \mathbf{C}$! (No need to evaluate right child).
- Node D: Left leaf is 0. MIN updates $\beta = 0$. Now $\alpha = \mathbf{3} \geq \beta = \mathbf{0} \implies \text{PRUNE RIGHT CHILD OF } \mathbf{D}$!
- $\text{Root Value } = \mathbf{3}$ (Move to B is optimal).

Topic 13.2: Master University Examination Solved Question Bank (10 Solved Questions)

Q1. Prove that A^* Search using tree-search is guaranteed to be optimal if the heuristic function $h(n)$ is admissible. (10 Marks)

Let G_2 be a sub-optimal goal state on the frontier, and $C^* = f(G^*) = g(G^*)$ be optimal goal cost. Suppose n is an unexpanded node on the path to optimal goal G^*. By admissibility, $h(n) \leq h^*(n)$, so $f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$. Since G_2 is sub-optimal, $f(G_2) = g(G_2) > C^* \geq f(n)$. Therefore, $f(n) \leq f(G_2)$ holds for all ancestors of optimal goal G^*, meaning A^* will always select and expand n before ever expanding sub-optimal goal G_2!

Q2. Compare Iterative Deepening Search (IDDFS) with Breadth-First Search (BFS) in terms of time and space complexity. (8 Marks)

BFS has time complexity $O(b^d)$ and space complexity $O(b^d)$, causing catastrophic out-of-memory errors on large trees. IDDFS combines the completeness and shallowest-goal optimality of BFS with the linear space complexity $O(bd)$ of DFS. In IDDFS, nodes at depth d are generated only once, depth $d-1$ twice, and the root $d+1$ times. The overhead factor is $\frac{b}{b-1}$, which is negligible for $b \geq 3$!

Q3. What is the Horizon Effect in Adversarial Game Playing? How is Quiescence Search used to mitigate it? (8 Marks)

The Horizon Effect occurs when an unavoidable catastrophic opponent move (e.g. losing a Queen) is pushed just beyond the search tree depth limit by executing stalling delaying moves (spurious checks). **Quiescence Search** mitigates this by extending the search beyond the fixed depth limit for "noisy" / unstable positions (e.g., capture sequences or checks) until the board reaches a quiescent (stable) state before evaluating the heuristic evaluation function.

Q4. Explain Constraint Satisfaction Problems (CSP). Trace the Backtracking Search with MRV and Forward Checking. (10 Marks)

A CSP is defined by Variables $X = \{X_1, \ldots, X_n\}$, Domains $D = \{D_1, \ldots, D_n\}$, and Constraints C. Standard backtracking is augmented by:
1. **Minimum Remaining Values (MRV / Most Constrained Variable):** Choose the variable with the fewest legal values remaining.
2. **Degree Heuristic:** Tie-breaker choosing variable with most constraints on remaining unassigned variables.
3. **Least Constraining Value (LCV):** Prefers the value that rules out the fewest choices for neighboring variables.
4. **Forward Checking:** Keeps track of remaining legal values for unassigned variables and terminates immediately if any domain becomes empty.

Q5. Explain Hill Climbing Search and its three primary failure modes. How does Simulated Annealing overcome them? (8 Marks)

Hill Climbing is a local search algorithm that iteratively moves in the direction of increasing elevation (greedy gradient ascent).
Failure Modes:
1. **Local Maxima:** A peak that is higher than neighboring states but lower than the global maximum.
2. **Ridges:** A sequence of local maxima where uphill moves are impossible in single coordinate directions.
3. **Plateaux / Shoulders:** Flat areas where the evaluation function is constant.
Simulated Annealing: Allows downhill moves with probability $P = e^{\Delta E/T}$. At high initial temperature T, it explores broadly; as $T \to 0$ (annealing schedule), it converges to the global maximum.

Q6. What is Genetic Algorithm (GA)? Explain Selection, Crossover, and Mutation with bit-string representations. (8 Marks)

A Genetic Algorithm is a stochastic local search inspired by natural evolution:
1. **Initial Population:** Set of candidate state bit-strings.
2. **Fitness Function:** Assigns higher numeric reproduction probability to better states.
3. **Selection:** Roulette wheel or tournament selection picks parent chromosomes proportional to fitness.
4. **Crossover:** Recombines parent bit-strings at random crossover points to produce offspring.
5. **Mutation:** Randomly flips individual bits with small probability p_m to maintain genetic diversity and prevent premature convergence.

Q7. Formulate the 8-Queens Problem as a State Space Search Problem. (6 Marks)

- **States:** Any arrangement of 0 to 8 queens on an 8×8 chessboard.
- **Initial State:** No queens on the board.
- **Actions:** Add a queen to any empty square in the next empty column such that no queen attacks another.
- **Goal Test:** 8 queens placed on the board with no two queens attacking each other (same row, column, or diagonal).
- **Path Cost:** Zero (only the final state matters).

Q8. Explain Iterative Deepening A^* (IDA*) and compare it with Memory-Bounded A^* (SMA*). (8 Marks)

IDA* is a memory-bounded heuristic search that uses depth-first search with an f-cost limit. Each iteration expands nodes until $f(n) > \text{limit}$. The next iteration's limit is the minimum f-value among all pruned nodes. It consumes only $O(bd)$ memory while retaining the cost-optimality of A^*! SMA* uses all available memory and prunes worst-performing subtrees while backing up values to parents.

Q9. Explain the Minimax Algorithm for Game Trees with Chance Nodes (Expectiminimax). (8 Marks)

In games with stochastic elements (e.g., Backgammon dice rolls), **Expectiminimax** adds *Chance Nodes* between MAX and MIN layers. The value of a Chance Node is the mathematical expected value: $\text{Expectivalue}(s) = \sum_s P(s) \cdot \text{Expectiminimax}(s)$. Alpha-Beta pruning can be adapted if bounds on evaluation functions are strictly known.

Q10. Calculate the Effective Branching Factor (b^*) if A^* search finds a solution at depth $d = 5$ after expanding $N = 150$ nodes. (6 Marks)

Using the polynomial formula $N + 1 = 1 + b^* + (b^*)^2 + (b^*)^3 + (b^*)^4 + (b^*)^5 = 151$:
For $b^* = 2$: $1 + 2 + 4 + 8 + 16 + 32 = 63 < 151$.
For $b^* = 2.5$: $1 + 2.5 + 6.25 + 15.625 + 39.0625 + 97.65625 = 162.09 > 151$.
Interpolating: $\mathbf{b^* \approx 2.45}$.

Topic 13.5: Local Search, Continuous Optimization & Genetic Algorithms

When the path to the goal is irrelevant and only the final state configuration matters (e.g. 8-Queens, VLSI circuit layout, protein folding), **Local Search Algorithms** operate on a single current state rather than maintaining extensive search tree paths.

Algorithm	Core Operational Mechanism	Space Complexity & Global Optimality
Hill-Climbing (Greedy Local Search)	Continuously moves in the direction of increasing elevation/objective value ($s' = \arg\max_{n \in \text{Neighbors}(s)} f(n)$). Terminates when no neighbor has higher value.	$\mathbf{O(1)}$ space; prone to getting trapped in local maxima, ridges, and plateau shoulders.
Simulated Annealing	Accepts uphill moves unconditionally ($\Delta E > 0$). Downhill moves are accepted with Boltzmann probability $\mathbf{P(\text{accept}) = e^{\Delta E / T}}$, where temperature T decreases according to cooling schedule $T(t)$.	$\mathbf{O(1)}$ space; asymptotically guaranteed to find global optimum if temperature decreases sufficiently slowly ($\sum_k T_k = \infty$).
Local Beam Search	Maintains k states in parallel. At each step, all successors of all k states are generated, and only the top k best successors across the entire pool are retained.	$\mathbf{O(k)}$ space; facilitates information sharing among parallel exploratory search trajectories.
Genetic Algorithms (GA)	Populations of candidate state chromosomes undergo fitness-proportional Selection (Roulette Wheel / Tournament), Crossover recombination, and point Mutation.	$\mathbf{O(\text{PopSize} \cdot L)}$; powerful global explorer across complex rugged discontinuous fitness landscapes.

🏛️ **Step-by-Step Solved Problem: Simulated Annealing Acceptance Probability Calculations**

An optimization problem minimizing cost E considers a neighbor with cost change $\Delta E = E_{\text{new}} - E_{\text{current}} = +15$ (worse by 15 units).

1. Calculate the acceptance probability at initial high temperature $T = 100$.
2. Calculate the acceptance probability at medium temperature $T = 20$.
3. Calculate the acceptance probability at freezing temperature $T = 1.0$.

Solution:

$$\mathbf{P = e^{-\Delta E / T}}$$

- At $T = 100$: $P = e^{-15/100} = e^{-0.15} \approx \mathbf{0.8607 = 86.07\%}$ (Very high exploratory freedom!).
- At $T = 20$: $P = e^{-15/20} = e^{-0.75} \approx \mathbf{0.4724 = 47.24\%}$ (Balanced exploration/exploitation).
- At $T = 1.0$: $P = e^{-15/1.0} = e^{-15} \approx \mathbf{3.059 \times 10^{-7} \approx 0.00003\%}$ (Pure greedy hill climbing!).

🏛️ Step-by-Step Solved Problem: Constraint Propagation via the AC-3 Algorithm

Let variables $X_1, X_2, X_3 \in \{1, 2, 3, 4\}$ be constrained by $X_1 < X_2$ and $X_2 < X_3$. Trace the **AC-3 (Arc Consistency 3) Algorithm**:

1. Initial domains: $D_1 = \{1, 2, 3, 4\}, D_2 = \{1, 2, 3, 4\}, D_3 = \{1, 2, 3, 4\}$.
2. Queue of Arcs = $\{(X_1, X_2), (X_2, X_1), (X_2, X_3), (X_3, X_2)\}$.
3. Pop (X_1, X_2): For $X_1 = 4$, no value in D_2 satisfies $4 < X_2$. Remove 4 from $D_1 \implies \mathbf{D_1 = \{1, 2, 3\}}$.
4. Pop (X_2, X_3): For $X_2 = 4$, no value in D_3 satisfies $4 < X_3$. Remove 4 from $D_2 \implies \mathbf{D_2 = \{1, 2, 3\}}$. Arc (X_1, X_2) re-added.
5. Pop (X_3, X_2): For $X_3 = 1$, no value in D_2 satisfies $X_2 < 1$. Remove 1 from $D_3 \implies \mathbf{D_3 = \{2, 3, 4\}}$.
6. Pop (X_1, X_2): For $X_1 = 3$, since $D_2 = \{1, 2, 3\}$, only $X_2 = 4$ could satisfy $3 < X_2$, but $4 \notin D_2$. Remove 3 from $D_1 \implies \mathbf{D_1 = \{1, 2\}}$.
7. Pop (X_2, X_1): For $X_2 = 1$, no value in $D_1 = \{1, 2\}$ satisfies $X_1 < 1$. Remove 1 from $D_2 \implies \mathbf{D_2 = \{2, 3\}}$.
8. Pop (X_3, X_2): For $X_3 = 2$, since $D_2 = \{2, 3\}$, no value satisfies $X_2 < 2$. Remove 2 from $D_3 \implies \mathbf{D_3 = \{3, 4\}}$.
9. Pop (X_1, X_2): For $X_1 = 2, X_2 = 3 \in D_2$ satisfies $2 < 3$. Valid!

$$\mathbf{\text{Final Arc-Consistent Domains: } D_1 = \{1, 2\}, \quad D_2 = \{2, 3\}, \quad D_3 = \{3, 4\}}$$

Q12. Prove that Depth-First Search with Graph Search is NOT optimal even for uniform step costs. (6 Marks)

Consider a graph where Start S has two edges: $S \xrightarrow{1} G$ (direct goal path, depth 1) and $S \xrightarrow{1} A \xrightarrow{1} G$ (depth 2). If DFS expands A first because of arbitrary node ordering, it adds A to the explored set and then reaches G via path $S \to A \to G$ with cost 2. When it backtracks to S, G is already in the explored set, so the direct optimal path $S \to G$ (cost 1) is never explored! Hence DFS is not cost-optimal.

Q13. What is the difference between Minimax with Alpha-Beta Pruning and SSS* (State Space Search) in Game Trees? (8 Marks)

• **Minimax with Alpha-Beta:** A depth-first traversal of the game tree that maintains single-path scalar bounds (α, β) and consumes only $O(bd)$ memory. It evaluates nodes in a fixed depth-first order.
• **SSS* (Stockman 1979):** A best-first branch-and-bound search that maintains an open list of game tree solution trees (sets of strategies). SSS* never expands any leaf that Alpha-Beta prunes, and can prune additional nodes that Alpha-Beta explores. However, SSS* requires exponential queue memory $O(b^{d/2})$, making it computationally impractical for deep game trees like Chess and Go!

Topic 13.6: Exhaustive Adversarial Game Theory & Alpha-Beta Deep Traces

Alpha-Beta Pruning Efficiency Theorems: - **Worst-Case Move Ordering (Worst-first):** Alpha-Beta examines every leaf node $\implies O(b^d)$ (No pruning advantage over standard Minimax). - **Ideal / Optimal Move Ordering (Best-first):** The effective branching factor is reduced from b to $\sqrt{b} \implies \mathbf{O(b^{d/2})}$. - *Profound Consequence:* With optimal move ordering, an Alpha-Beta search can search **twice as deep** in the same compute time as standard Minimax (e.g., searching 12 plies instead of 6 plies in Chess)!

🏛️ Step-by-Step Solved Problem: Full 3-Ply Alpha-Beta Pruning Tree Trace

Consider a 3-ply Minimax tree where Root is MAX, Level 1 is MIN (B, C), Level 2 is MAX (D, E, F, G), and Level 3 has 8 terminal leaves:

- Node D leaves: $[4, 6] \implies \text{Node } D = \max(4, 6) = 6$.
- Node E leaves: $[7, 9] \implies \text{Node } E = \max(7, 9) = 9$.
- Node B (MIN of D, E): $\beta = \min(6, 9) = 6$. Updates Root MAX: $\alpha = 6$.
- Node F left leaf: 1. Since Node F is MAX, $F \geq 1$. Node F right leaf: $2 \implies \text{Node } F = 2$.
- Node C (MIN): Updates $\beta = 2$.
- Now at Node C, we have $\alpha = \mathbf{6} \geq \beta = \mathbf{2} \implies \text{PRUNE NODE G AND ALL ITS LEAVES COMPLETELY!}$
- $\text{Root Optimal Value } = \mathbf{6} \quad (\text{Optimal Move: } \mathbf{A \to B})$.

Q14. Explain Transposition Tables in Game Tree Search. How do Zobrist Hashing keys operate? (8 Marks)

In games like Chess, different move orders can lead to the identical board position (Transposition: $1.e4\ e5\ 2.Nf3\ Nc6$ vs $1.Nf3\ Nc6\ 2.e4\ e5$). A **Transposition Table** is a high-speed hash map caching previously evaluated board states.
Zobrist Hashing: Assigns a random 64-bit integer to each `(piece, square)` tuple. A board position's hash is computed by XORing ($\oplus$) all present piece keys. When a move occurs, updating the hash takes $O(1)$ XOR operations ($\text{Hash}' = \text{Hash} \oplus \text{Key}_{\text{old}} \oplus \text{Key}_{\text{new}}$), enabling millions of board lookups per second!

Topic 13.7: Advanced Adversarial Search & Monte Carlo Tree Search (MCTS)

In ultra-large state space games where evaluation functions cannot be hand-crafted (e.g. Go, Chess variants), **Monte Carlo Tree Search (MCTS)** uses stochastic rollouts to evaluate positions.

Figure 2.1: The Four Discrete Phases of Monte Carlo Tree Search (MCTS / AlphaGo)

The Upper Confidence Bound for Trees (UCT) Formula:

$$\text{UCT}(\mathbf{v_i}, \mathbf{v}) = \frac{\mathbf{Q(v_i)}}{\mathbf{N(v_i)}} + \mathbf{c}\sqrt{\frac{\ln \mathbf{N(v)}}{\mathbf{N(v_i)}}} = \text{Exploitation Term} + \text{Exploration Term}$$

Where $Q(v_i)$ is total simulation reward, $N(v_i)$ is child visit count, $N(v)$ is parent visit count, and $c = \sqrt{2}$ is the exploration parameter balancing exploitation of winning moves with exploration of rarely visited branches.

🏛️ **Step-by-Step Solved Problem: MCTS UCT Calculation**

A parent node v has been visited $N(v) = 100$ times. It has two children A and B:

- Child A: Visited $N(A) = 60$ times with total wins $Q(A) = 45$.
- Child B: Visited $N(B) = 40$ times with total wins $Q(B) = 28$.

$$\text{UCT}(A) = \frac{45}{60} + \sqrt{2}\sqrt{\frac{\ln 100}{60}} = 0.75 + 1.414\sqrt{\frac{4.605}{60}} = 0.75 + 1.414(0.277) = 0.75 + 0.392 = \mathbf{1.142}$$

$$\text{UCT}(B) = \frac{28}{40} + \sqrt{2}\sqrt{\frac{\ln 100}{40}} = 0.70 + 1.414\sqrt{\frac{4.605}{40}} = 0.70 + 1.414(0.339) = 0.70 + 0.479 = \mathbf{1.179}$$

Selection Decision: Choose Child $\mathbf{B}$ because its exploration potential (1.179) outweighs Child $\mathbf{A}$ (1.142)!

Topic 13.8: Complete Step-by-Step Solved Problem Bank (Part III)

🏛️ Solved Numerical 3: IDA* (Iterative Deepening A*) Search Execution Trace

Consider a search tree with initial state S and goal G with branch costs and heuristics:

- $S \xrightarrow{2} A\ (h=5),\ S \xrightarrow{3} B\ (h=4)$.
- $A \xrightarrow{4} C\ (h=3),\ A \xrightarrow{5} D\ (h=2)$.
- $B \xrightarrow{2} E\ (h=3),\ B \xrightarrow{6} G\ (h=0)$.
- $C \xrightarrow{2} G\ (h=0)$.

IDA* Iterations:

1. **Iteration 1 (Cutoff = $f(S) = g(S) + h(S) = 0 + 6 = 6$):**
   - Expand S: $f(A) = 2 + 5 = 7 > 6$ (Pruned, min cutoff = 7); $f(B) = 3 + 4 = 7 > 6$ (Pruned, min cutoff = 7).
   - Threshold for Iteration 2 is set to $\mathbf{7}$.
2. **Iteration 2 (Cutoff = 7):**
   - Expand S: $A\,(f = 7 \leq 7), B\,(f = 7 \leq 7)$.
   - Expand A: $f(C) = 2 + 4 + 3 = 9 > 7$ (Pruned, cutoff=9); $f(D) = 2 + 5 + 2 = 9 > 7$ (Pruned, cutoff=9).
   - Expand B: $f(E) = 3 + 2 + 3 = 8 > 7$ (Pruned, cutoff=8); $f(G) = 3 + 6 + 0 = 9 > 7$ (Pruned, cutoff=9).
   - Next threshold is $\min(9, 9, 8, 9) = \mathbf{8}$.
3. **Iteration 3 (Cutoff = 8):**
   - Expand $S \to B \to E$: Successors of $E \ldots$
   - Next threshold = $\mathbf{8}$. Goal G reached via path $S \to C \to G$ with cost $\mathbf{8}$!

Q17. Explain Simulated Annealing Cooling Schedules and the Metropolis Criterion. (8 Marks)

The **Metropolis Criterion** dictates that downhill moves (cost improvements) are accepted with probability 1.0, while uphill moves ($\Delta E > 0$) are accepted with probability $P = e^{-\Delta E / T}$.

Cooling Schedules:

1. *Linear Cooling:* $T(t) = T_0 - \alpha t$.
2. *Geometric / Exponential Cooling:* $T(t) = T_0 \cdot \alpha^t$ (where $\alpha \in [0.80, 0.99]$).
3. *Logarithmic Cooling (Geman & Geman):* $T(t) = \frac{C}{\ln(1+t)}$. This is the only schedule mathematically proven to converge to the global optimum, but requires astronomically many iterations!

MODULE 3 OF 5

Module 3: Knowledge Representation & Logic

Topics 14 to 22 • Wumpus World, Propositional CNF, First-Order Logic & Resolution

Module 3 Table of Contents (Topics 14 to 22)

Topic 14 & 15: Knowledge-Based Agents & The Wumpus World

A **Knowledge-Based Agent** maintains an internal **Knowledge Base (KB)** consisting of sentences in a formal representation language. It interacts with the KB via two fundamental operations:

$$\text{TELL}(\mathbf{KB}, \alpha) \quad \text{and} \quad \text{ASK}(\mathbf{KB}, \alpha)$$

The Wumpus World Formal Specification

- **Grid:** 4×4 grid of rooms with Start at $[1, 1]$.
- **Hazards:** Bottomless Pits (P) with probability 0.2 in each room; One deadly Wumpus (W).
- **Reward:** Gold glitter (G) at random location.
- **Percepts:** `[Stench, Breeze, Glitter, Bump, Scream]`.
  - In rooms adjacent to Wumpus $\implies$ *Stench*: $S_{x,y} \iff (W_{x-1,y} \lor W_{x+1,y} \lor W_{x,y-1} \lor W_{x,y+1})$.
  - In rooms adjacent to Pits $\implies$ *Breeze*: $B_{x,y} \iff (P_{x-1,y} \lor P_{x+1,y} \lor P_{x,y-1} \lor P_{x,y+1})$.

Topic 16 to 18: Propositional Entailment, CNF & Resolution

Entailment vs. Logical Equivalence: - **Entailment ($\alpha \models \beta$):** Sentence β follows logically from α if and only if in every model where α is true, β is also true ($M(\alpha) \subseteq M(\beta)$). - **Proof by Contradiction (Refutation):**

$$\mathbf{KB} \models \alpha \iff (\mathbf{KB} \land \neg\alpha) \text{ is UNSATISFIABLE (generates empty clause } \square)$$

The 6-Step Algorithm to Convert Any Propositional Sentence into Conjunctive Normal Form (CNF)

1. **Eliminate Equivalence ($\leftrightarrow$):** Replace $\alpha \leftrightarrow \beta$ with $(\alpha \rightarrow \beta) \land (\beta \rightarrow \alpha)$.
2. **Eliminate Implication ($\rightarrow$):** Replace $\alpha \rightarrow \beta$ with $\neg\alpha \lor \beta$.
3. **Move Negation Inward (De Morgan's Laws):** $\neg(\alpha \land \beta) \equiv \neg\alpha \lor \neg\beta, \neg(\alpha \lor \beta) \equiv \neg\alpha \land \neg\beta, \neg\neg\alpha \equiv \alpha$.
4. **Distribute $\lor$ over $\land$:** Replace $\alpha \lor (\beta \land \gamma)$ with $(\alpha \lor \beta) \land (\alpha \lor \gamma)$.
5. **Flatten Nested Conjunctions/Disjunctions:** $(A \lor B) \lor C \equiv (A \lor B \lor C)$.
6. **Split into Clauses:** Each conjunct becomes a separate clause in the set.

Topic 19 to 22: First-Order Logic (FOL), Unification & Resolution Refutation

First-Order Logic adds **Objects, Relations (Predicates), Functions, and Quantifiers (∀, ∃)**.

English Statement	First-Order Logic Translation	Quantifier Rule
"Every student loves AI."	$\forall x(\text{Student}(x) \rightarrow \text{Loves}(x, \text{AI}))$	$\forall$ typically pairs with $\rightarrow$.
"Some student loves AI."	$\exists x(\text{Student}(x) \land \text{Loves}(x, \text{AI}))$	$\exists$ typically pairs with $\land$.
"No person likes snakes."	$\forall x(\text{Person}(x) \rightarrow \neg\text{Likes}(x, \text{Snakes}))$	$\neg\exists x \cdots \equiv \forall x \neg \ldots$

🏛️ Step-by-Step Unification Algorithm & Most General Unifier (MGU)

Find $\text{UNIFY}(P_1, P_2)$ for:

1. $P_1 = \text{Knows}(\text{John}, x)$ and $P_2 = \text{Knows}(y, \text{Bill}) \implies \theta = \{\mathbf{y}/\text{John}, \mathbf{x}/\text{Bill}\}$.
2. $P_1 = \text{Knows}(\text{John}, x)$ and $P_2 = \text{Knows}(x, \text{Bill}) \implies$ Fail (Standardize variables apart first!).
3. $P_1 = \text{Likes}(x, \text{Father}(x))$ and $P_2 = \text{Likes}(y, y) \implies$ Occur Check Failure! (y cannot unify with $\text{Father}(y)$).

🏛️ Complete Step-by-Step FOL Resolution Refutation Proof

Given Axioms:

1. "Marcus was a man." $\implies \text{Man}(\text{Marcus})$
2. "Marcus was a Pompeian." $\implies \text{Pompeian}(\text{Marcus})$
3. "All Pompeians were Romans." $\implies \forall x(\text{Pompeian}(x) \rightarrow \text{Roman}(x)) \implies \neg\text{Pompeian}(x) \lor \text{Roman}(x)$
4. "Caesar was a ruler." $\implies \text{Ruler}(\text{Caesar})$
5. "All Romans were either loyal to Caesar or hated him." $\implies \forall x(\text{Roman}(x) \rightarrow \text{Loyal}(x, \text{Caesar}) \lor \text{Hate}(x, \text{Caesar})) \implies \neg\text{Roman}(x) \lor \text{Loyal}(x, \text{Caesar}) \lor \text{Hate}(x, \text{Caesar})$
6. "Everyone is loyal to someone." $\implies \forall x \exists y \text{Loyal}(x, y) \implies \text{Loyal}(x, f(x))$ (Skolem function)
7. "Men only try to assassinate rulers they aren't loyal to." $\implies \forall x \forall y(\text{Man}(x) \land \text{Ruler}(y) \land \text{TryAssassinate}(x, y) \rightarrow \neg\text{Loyal}(x, y)) \implies \neg\text{Man}(x) \lor \neg\text{Ruler}(y) \lor \neg\text{TryAssassinate}(x, y) \lor \neg\text{Loyal}(x, y)$
8. "Marcus tried to assassinate Caesar." $\implies \text{TryAssassinate}(\text{Marcus}, \text{Caesar})$

Query to Prove: "Did Marcus hate Caesar?" ($\text{Hate}(\text{Marcus}, \text{Caesar})$)

Proof by Refutation: Add negated query $\neg\text{Hate}(\text{Marcus}, \text{Caesar})$:

1. Resolve $\neg\text{Hate}(\text{Marcus}, \text{Caesar})$ with Clause 5 $\{x/\text{Marcus}\} \implies \mathbf{C_9} : \neg\text{Roman}(\text{Marcus}) \lor \text{Loyal}(\text{Marcus}, \text{Caesar})$.
2. Resolve C_9 with Clause 3 $\{x/\text{Marcus}\} \implies \mathbf{C_{10}} : \neg\text{Pompeian}(\text{Marcus}) \lor \text{Loyal}(\text{Marcus}, \text{Caesar})$.
3. Resolve C_{10} with Clause 2 $\implies \mathbf{C_{11}} : \text{Loyal}(\text{Marcus}, \text{Caesar})$.
4. Resolve C_{11} with Clause 7 $\{x/\text{Marcus}, y/\text{Caesar}\} \implies$ $\mathbf{C_{12}} : \neg\text{Man}(\text{Marcus}) \lor \neg\text{Ruler}(\text{Caesar}) \lor \neg\text{TryAssassinate}(\text{Marcus}, \text{Caesar})$.
5. Resolve C_{12} with Clause 1 $\implies \mathbf{C_{13}} : \neg\text{Ruler}(\text{Caesar}) \lor \neg\text{TryAssassinate}(\text{Marcus}, \text{Caesar})$.
6. Resolve C_{13} with Clause 4 $\implies \mathbf{C_{14}} : \neg\text{TryAssassinate}(\text{Marcus}, \text{Caesar})$.
7. Resolve C_{14} with Clause 8 $\implies \square$ (EMPTY CLAUSE - CONTRADICTION!).

Q.E.D. Marcus hated Caesar is strictly proven!

Topic 22.2: Master University Examination Solved Question Bank (10 Solved Questions)

Q1. Prove that Modus Ponens is sound. (6 Marks)

Modus Ponens states: From α and $\alpha \rightarrow \beta$, infer β. In truth table semantics: Whenever α is True and $\alpha \rightarrow \beta$ is True, β must be True (since if β were False, $\alpha \rightarrow \beta$ would be False, contradicting the premise). Thus, Modus Ponens preserves truth in all possible models.

Q2. Convert the sentence $\neg \forall x(\text{Dog}(x) \rightarrow \exists y(\text{Cat}(y) \wedge \text{Chases}(x, y)))$ into Skolemized CNF. (10 Marks)

1. Move negation inward: $\exists x \neg(\neg \text{Dog}(x) \vee \exists y(\text{Cat}(y) \wedge \text{Chases}(x, y))) \implies \exists x(\text{Dog}(x) \wedge \forall y(\neg \text{Cat}(y) \vee \neg \text{Chases}(x, y)))$.
2. Skolemize existential variable x with Skolem constant D: $\text{Clause 1: } \text{Dog}(\mathbf{D}), \text{Clause 2: } \neg \text{Cat}(\mathbf{y}) \vee \neg \text{Chases}(\mathbf{D}, \mathbf{y})$.

Q3. Explain Forward Chaining and Backward Chaining for Horn Clauses. Compare their computational efficiencies. (8 Marks)

• **Forward Chaining (Data-Driven):** Starts from known facts in the KB and iteratively fires rules whose premises are satisfied, adding new conclusions until the query is reached. Complete and runs in $O(\text{size of KB})$ time for definite clauses.
• **Backward Chaining (Goal-Driven):** Starts from the goal query and recursively searches backwards for rules whose conclusions match the goal, resolving sub-goals against facts. Avoids generating facts irrelevant to the goal.

Q4. What is the Occur Check in the Unification Algorithm? Why is it crucial? (8 Marks)

The Occur Check verifies whether a variable v appears inside the term t before binding v/t. Example: Unifying x with $f(x)$. If permitted, it produces an infinite recursive term $x = f(f(f(\ldots)))$, which leads to non-terminating loops and logical unsoundness in automated theorem provers! (Standard Prolog omits occur-check for speed, leading to occasional unsound derivations).

Q5. Explain the Frame Problem, Qualification Problem, and Ramification Problem in Knowledge Representation. (8 Marks)

1. **Frame Problem:** Representing what remains *unchanged* in the world when an action is executed without explicitly listing millions of non-effects.
2. **Qualification Problem:** The impossibility of listing all infinite preconditions required for an action to succeed (e.g. car starting requires battery, gas, no potato in tailpipe, no asteroid strike).
3. **Ramification Problem:** Handling implicit side-effects (indirect consequences) of actions without writing separate explicit rules for each.

Q6. Convert the proposition $(A \wedge B) \rightarrow (C \vee D)$ into Conjunctive Normal Form (CNF). (6 Marks)

1. Eliminate implication: $\neg(A \wedge B) \vee (C \vee D)$.
2. De Morgan's: $(\neg A \vee \neg B) \vee (C \vee D)$.
3. Flatten disjunction: $(\neg \mathbf{A} \vee \neg \mathbf{B} \vee \mathbf{C} \vee \mathbf{D})$ (Single valid CNF clause!).

Q7. Explain Semantic Networks and Frames as Knowledge Representation schemes. (8 Marks)

• **Semantic Networks:** Represent knowledge as directed graphs where nodes represent concepts/objects and edges represent binary relations (e.g., `is-a`, `has-a`, `part-of`). Enables taxonomic property inheritance.
• **Frames (Minsky):** Structured data structures containing named *slots* and associated *fillers* (values, default values, or procedural attachment methods/demons).

Q8. What is Non-Monotonic Logic? Explain Default Reasoning and Circumscription. (8 Marks)

In classical monotonic logic, adding new axioms can never invalidate previously derived theorems. In **Non-Monotonic Logic**, conclusions can be retracted when new evidence arrives (e.g. "Tweety is a bird $\implies$ Tweety flies"; Learning "Tweety is a penguin" retracts "Tweety flies"). **Default Logic** introduces default inference rules $\frac{\alpha : \beta}{\gamma}$. **Circumscription** formalizes the assumption that things are normal unless explicitly stated otherwise.

Q9. Explain the DPLL (Davis-Putnam-Logemann-Loveland) Satisfiability Algorithm. (8 Marks)

DPLL is a recursive backtracking search for checking SAT over CNF clauses:
1. Early Termination (SAT if all clauses true; Backtrack if any clause empty).
2. Pure Literal Rule: Literals appearing with only one polarity across all clauses are assigned that polarity immediately.
3. Unit Propagation: Unit clauses force immediate deterministic variable assignments.
4. Splitting: Branch recursively on $X = \text{True}$ and $X = \text{False}$.

Q10. Explain Ontology Engineering and the Role of Description Logics (OWL). (8 Marks)

An **Ontology** formally defines the concepts, entities, categories, and relationships within a domain. **Description Logics (DLs)** (like $\mathcal{SHIQ}, \mathcal{SROIQ}$) are decidable fragments of First-Order Logic providing the theoretical foundation for Semantic Web standards (OWL / RDF). DL systems provide automated subsumption checking ($C \sqsubseteq D$) and consistency checking.

Topic 22.5: Advanced Automated Theorem Proving, Semantic Tableaux & Ontologies

🏛️ Step-by-Step Solved Proof: Complete Forward Chaining Trace on Definite Horn Clauses

Given Knowledge Base of Horn Clauses:

1. $A \land B \to C$
2. $C \land D \to E$
3. $B \land E \to F$
4. $G \land A \to D$
5. Known Facts: $\mathbf{A}$, $\mathbf{B}$, $\mathbf{G}$

Query: Prove if goal F is entailed ($\text{KB} \models F$).

Forward Chaining Execution Table:

Iteration	Rule Fired	Satisfied Premises	New Fact Inferred	Known Facts Pool
0	Initial Facts	—	A, B, G	$\{A, B, G\}$
1	Rule 4 ($G \land A \to D$)	$G \in KB, A \in KB$	D	$\{A, B, G, D\}$
2	Rule 1 ($A \land B \to C$)	$A \in KB, B \in KB$	C	$\{A, B, G, D, C\}$
3	Rule 2 ($C \land D \to E$)	$C \in KB, D \in KB$	E	$\{A, B, G, D, C, E\}$
4	Rule 3 ($B \land E \to F$)	$B \in KB, E \in KB$	F	$\{\mathbf{A}, \mathbf{B}, \mathbf{G}, \mathbf{D}, \mathbf{C}, \mathbf{E}, \mathbf{F}\}$

Conclusion: Goal $\mathbf{F}$ is derived in linear time $\mathbf{O(N)}$!

🏛️ The Method of Analytic Semantic Tableaux for First-Order Logic

An alternative to Resolution is the **Semantic Tableaux (Tree Proof) Method**, which builds a tree of signed formulas to prove that $\text{KB} \land \neg\text{Query}$ is closed (every branch contains a contradiction P and $\neg P$):

- **α-Rules (Conjunctive):** Add both conjuncts to the current branch without splitting ($A \land B \implies A, B$).
- **β-Rules (Disjunctive):** Split the current branch into two child branches ($A \lor B \implies \text{Left: } A, \text{ Right: } B$).
- **γ-Rules (Universal):** Instantiate $\forall x P(x)$ with any arbitrary ground term t ($P(t)$).
- **δ-Rules (Existential):** Instantiate $\exists x P(x)$ with a fresh Skolem constant c ($P(c)$).

Q11. Explain Description Logics (DL) Concepts, Roles, and TBox vs. ABox. (8 Marks)

Description Logics (DLs) form the formal logical foundation for the Semantic Web (OWL-DL) and modern knowledge graphs:
• **Concepts (Unary Predicates):** Sets of individuals (e.g., $\text{Student}, \text{Course}$).
• **Roles (Binary Relations):** Relationships between individuals (e.g., $\text{enrolledIn}, \text{teaches}$).
• **TBox (Terminological Box):** The schema containing conceptual definitions, axioms, and subsumption hierarchies ($\text{CSEStudent} \equiv \text{Student} \sqcap \exists \text{enrolledIn}.\text{CSECourse}$).
• **ABox (Assertional Box):** Concrete assertions about specific named individuals ($\text{Student}(\text{Shaswat}), \text{enrolledIn}(\text{Shaswat}, \text{CS24307})$).

Q12. What is Default Reasoning and Reiter's Default Logic? (8 Marks)

Default logic allows an agent to draw plausible inferences in the absence of contrary evidence using default rules:

$$\frac{\alpha : \beta}{\gamma}$$

Where α is the *prerequisite*, β is the *justification* (must be consistent with the KB, $\neg\beta \notin KB$), and γ is the *conclusion*.
Example (Bird Flight): $\frac{\text{Bird}(x) : \text{Flies}(x)}{\text{Flies}(x)}$. If $\text{Bird}(\text{Tweety})$ is known and $\neg\text{Flies}(\text{Tweety})$ is NOT in the KB, the agent infers $\text{Flies}(\text{Tweety})$. If it later learns $\text{Penguin}(\text{Tweety}) \land (\text{Penguin}(x) \to \neg\text{Flies}(x))$, the justification fails and the conclusion is retracted automatically!

Topic 22.6: Advanced Resolution Strategies & Knowledge Compilation

Resolution Strategy	Algorithmic Restriction	Completeness & Efficiency
Unit Resolution	At least one of the two parent clauses must be a *unit clause* (containing exactly one literal).	Incomplete in general; complete and runs in $O(N)$ time for Horn KBs.
Input Resolution	At least one parent clause must come from the original input KB or negated query (never two derived clauses).	Incomplete in general; equivalent in power to Unit Resolution.
Linear Resolution	Each step resolves the most recently derived clause with either an input clause or a previous ancestor clause.	**Refutation Complete**; basis for Prolog's SLD resolution.
Set of Support (SOS)	Every resolution step must involve at least one clause derived from the negated query (the set of support).	**Refutation Complete**; prevents wasteful resolution among mutually consistent background KB axioms.

🏛️ **Step-by-Step Solved Proof: The "Customs Official" Full Refutation Proof Tree**

Axioms:

1. $C_1 : \neg\text{Official}(x) \lor \neg\text{Enters}(y) \lor \text{VIP}(y) \lor \text{Searches}(x, y)$
2. $C_2 : \text{Smuggler}(A)$ (Skolem constant A)
3. $C_3 : \text{Enters}(A)$
4. $C_4 : \neg\text{Searches}(z, A) \lor \text{Smuggler}(z)$
5. $C_5 : \neg\text{Smuggler}(x) \lor \neg\text{VIP}(x)$
6. $C_6 : \text{Official}(B)$ (Official B exists)

Query: Prove $\exists w(\text{Official}(w) \land \text{Smuggler}(w))$. Negated Query: $C_7 : \neg\text{Official}(w) \lor \neg\text{Smuggler}(w)$.

Resolution Proof Steps:

1. Resolve C_2 $(\text{Smuggler}(A))$ with C_5 $\{x/A\} \implies \mathbf{C_8 : \neg\text{VIP}(A)}$.
2. Resolve C_1 with C_3 $\{y/A\} \implies \mathbf{C_9 : \neg\text{Official}(x) \lor \text{VIP}(A) \lor \text{Searches}(x, A)}$.
3. Resolve C_9 with $C_8 \implies \mathbf{C_{10} : \neg\text{Official}(x) \lor \text{Searches}(x, A)}$.
4. Resolve C_{10} with C_4 $\{z/x\} \implies \mathbf{C_{11} : \neg\text{Official}(x) \lor \text{Smuggler}(x)}$.
5. Resolve C_{11} with C_6 $\{x/B\} \implies \mathbf{C_{12} : \text{Smuggler}(B)}$.
6. Resolve C_{12} with C_7 $\{w/B\} \implies \mathbf{C_{13} : \neg\text{Official}(B)}$.
7. Resolve C_{13} with C_6 $(\text{Official}(B)) \implies \square$ (EMPTY CLAUSE - CONTRADICTION!).

Q.E.D. Strictly proven that some official must be a smuggler!

Topic 22.7: Advanced Knowledge Graphs & Description Logic Semantics

Description Logic ($\mathcal{ALC}$) Syntax & Semantics: - Top (Universal Concept): $\top \implies \Delta^{\mathcal{I}}$ (All domain elements) - Bottom (Empty Concept): $\bot \implies \emptyset$ - Conjunction: $(C \sqcap D)^{\mathcal{I}} = C^{\mathcal{I}} \cap D^{\mathcal{I}}$ - Disjunction: $(C \sqcup D)^{\mathcal{I}} = C^{\mathcal{I}} \cup D^{\mathcal{I}}$ - Negation: $(\neg C)^{\mathcal{I}} = \Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}$ - Universal Role Restriction: $(\forall R.C)^{\mathcal{I}} = \{x \in \Delta^{\mathcal{I}} \mid \forall y((x, y) \in R^{\mathcal{I}} \to y \in C^{\mathcal{I}})\}$ - Existential Role Restriction: $(\exists R.C)^{\mathcal{I}} = \{x \in \Delta^{\mathcal{I}} \mid \exists y((x, y) \in R^{\mathcal{I}} \land y \in C^{\mathcal{I}})\}$

🏛️ **Step-by-Step Solved Problem: Description Logic Subsumption Proof**

Prove that $\text{Mother} \sqsubseteq \text{Parent}$ given the TBox axioms:

1. $\text{Parent} \equiv \text{Human} \sqcap \exists \text{hasChild}.\text{Human}$
2. $\text{Mother} \equiv \text{Woman} \sqcap \exists \text{hasChild}.\text{Human}$
3. $\text{Woman} \sqsubseteq \text{Human}$

Proof Trace:

- Let $x \in \text{Mother}^{\mathcal{I}} \implies x \in \text{Woman}^{\mathcal{I}} \wedge x \in (\exists \text{hasChild}.\text{Human})^{\mathcal{I}}$.
- Since $\text{Woman}^{\mathcal{I}} \subseteq \text{Human}^{\mathcal{I}}$ (Axiom 3), we have $x \in \text{Human}^{\mathcal{I}}$.
- Therefore, $x \in \text{Human}^{\mathcal{I}} \wedge x \in (\exists \text{hasChild}.\text{Human})^{\mathcal{I}} \implies x \in \text{Parent}^{\mathcal{I}}$.
- $\text{Q.E.D. Subsumption Mother} \sqsubseteq \text{Parent holds in all models!}$

Topic 22.8: Complete Step-by-Step Solved Proof Bank (Part III)

🏛️ **Step-by-Step Solved Proof: The "Customs and Smuggler" Full Resolution Refutation Tree**

Axioms:

1. $\neg \text{Official}(x) \vee \neg \text{Enters}(y) \vee \text{VIP}(y) \vee \text{Searches}(x, y)$
2. $\text{Smuggler}(A) \wedge \text{Enters}(A) \wedge (\neg \text{Searches}(z, A) \vee \text{Smuggler}(z))$ (Skolem constant A)
3. $\neg \text{Smuggler}(x) \vee \neg \text{VIP}(x)$
4. $\text{Official}(O_1)$

Refutation Steps:

1. Resolve $\text{Smuggler}(A)$ with Clause 3 $\{x/A\} \implies \neg \text{VIP}(\mathbf{A})$.
2. Resolve Clause 1 with $\text{Enters}(A)$ $\{y/A\} \implies \neg \text{Official}(x) \vee \text{VIP}(A) \vee \text{Searches}(x, A)$.
3. Resolve with $\neg \text{VIP}(A) \implies \neg \text{Official}(x) \vee \text{Searches}(x, A)$.
4. Resolve with Clause 2 $(\neg \text{Searches}(z, A) \vee \text{Smuggler}(z))$ $\{z/x\} \implies \neg \text{Official}(\mathbf{x}) \vee \text{Smuggler}(\mathbf{x})$.
5. Resolve with $\text{Official}(O_1) \implies \text{Smuggler}(\mathbf{O_1})$.
6. Resolve with Negated Query clause $(\neg \text{Official}(O_1) \vee \neg \text{Smuggler}(O_1)) \implies \square$ (EMPTY CLAUSE).

$$\text{Q.E.D. Proof complete!}$$

🏛️ **Step-by-Step Solved Problem: Situation Calculus Axiomatization**

In Situation Calculus, actions take an agent from situation s to situation $do(a, s)$:

- **Possibility Axiom (Preconditions for Pickup):**

$$\text{Poss}(\text{PickUp}(\mathbf{x}), \mathbf{s}) \iff \text{Clear}(\mathbf{x}, \mathbf{s}) \wedge \text{At}(\text{Robot}, \mathbf{x}, \mathbf{s}) \wedge \text{ArmEmpty}(\mathbf{s})$$

- **Successor-State Axiom (Effect on Holding):**

$$\text{Holding}(\mathbf{x}, \mathbf{do}(\mathbf{a}, \mathbf{s})) \iff (\mathbf{a} = \text{PickUp}(\mathbf{x})) \vee (\text{Holding}(\mathbf{x}, \mathbf{s}) \wedge \mathbf{a} \neq \text{Release}(\mathbf{x}))$$

Q16. Explain Knowledge Base Consistency and Model Checking using Truth Tables. (8 Marks)

A Knowledge Base KB is **consistent (satisfiable)** if there exists at least one truth assignment (model) under which all sentences in KB evaluate to True. **Model Checking** enumerates all 2^n interpretations of the n proposition symbols in a truth table. For each row where $KB = \text{True}$, it verifies if the query sentence $\alpha = \text{True}$. If α is True in every model where KB is True, then $KB \models \alpha$ (sound and complete, but exponential time $O(2^n)$).

Topic 22.9: Modal Logic, Epistemic Reasoning & Dempster-Shafer Theory

Formal Logic System	Modal Operators & Axioms	AI Multi-Agent Application
Epistemic Logic ($S5$)	Knowledge operator $K_i\phi$ ("Agent i knows ϕ"). Axioms: $K_i\phi \to \phi$ (Truth), $K_i\phi \to K_iK_i\phi$ (Positive Introspection), $\neg K_i\phi \to K_i\neg K_i\phi$ (Negative Introspection).	Distributed systems consensus, multi-agent common knowledge ($C\phi$), Muddy Children puzzle.
Temporal Logic (LTL / CTL)	LTL operators: $\mathbf{G}\phi$ (Always), $\mathbf{F}\phi$ (Eventually), $\mathbf{X}\phi$ (Next), $\phi\mathbf{U}\psi$ (Until). CTL adds path quantifiers $\mathbf{A}$ (All paths), $\mathbf{E}$ (Exists path).	Model checking automated safety verification of autonomous flight control and medical robotics software.
Dempster-Shafer Theory of Evidence	Mass function $m(A) \in [0,1]$ over power set 2^Θ. Belief function $\text{Bel}(A) = \sum_{B \subseteq A} m(B)$, Plausibility $\text{Pl}(A) = 1 - \text{Bel}(\neg A)$. Dempster's Rule of Combination: $m_1 \oplus m_2(A) = \frac{\sum_{B \cap C = A} m_1(B)m_2(C)}{1-K}$.	Sensor fusion with epistemic ignorance (distinguishes total ignorance from equal probability!).

🏛️ Step-by-Step Solved Problem: Dempster's Rule of Combination in Sensor Fusion

Two medical sensors diagnose a patient for three mutually exclusive conditions $\Theta = \{\text{Flu } (F), \text{Cold } (C), \text{Pneumonia } (P)\}$:

- Sensor 1: $m_1(\{F\}) = 0.6,\ m_1(\Theta) = 0.4$.
- Sensor 2: $m_2(\{F, C\}) = 0.7,\ m_2(\Theta) = 0.3$.

Dempster Combination Matrix:

$m_1 \backslash m_2$	$m_2(\{F,C\}) = 0.7$	$m_2(\Theta) = 0.3$
$m_1(\{F\}) = 0.6$	$\{F\} \cap \{F,C\} = \{F\} \implies 0.42$	$\{F\} \cap \Theta = \{F\} \implies 0.18$
$m_1(\Theta) = 0.4$	$\Theta \cap \{F,C\} = \{F,C\} \implies 0.28$	$\Theta \cap \Theta = \Theta \implies 0.12$

No empty set intersections ($K = 0$). Combined masses:

$$\mathbf{m_{1,2}(\{F\}) = 0.42 + 0.18 = 0.60} \qquad \mathbf{m_{1,2}(\{F,C\}) = 0.28} \qquad \mathbf{m_{1,2}(\Theta) = 0.12}$$

$$\mathbf{\text{Belief in Flu: } \text{Bel}(\{F\}) = m_{1,2}(\{F\}) = \mathbf{0.60} \qquad \text{Plausibility of Flu: } \text{Pl}(\{F\}) = 0.60 + 0.28 + 0.12 = \mathbf{1.00}}$$

Q17. Explain the Resolution Refutation Proof with Equality (Paramodulation and Demodulation). (8 Marks)

Standard resolution handles predicate symbols but cannot natively reason about equality axioms ($x = x, x = y \to y = x, x = y \wedge y = z \to x = z$).
• **Paramodulation:** A specialized inference rule that incorporates equality: From clause $(l = r \vee C)$ and clause $(P(t) \vee D)$, where a subterm of t unifies with l under $\theta = \text{UNIFY}(t|_p, l)$, infer $(P(t[r\theta]_p) \vee C\theta \vee D\theta)$.
• **Demodulation:** A deterministic rewriting rule that uses unit equality clauses $l = r$ to simplify terms in other clauses to a canonical normal form, preventing exponential branching in equational theorem provers like Otter and Vampire!

Topic 22.10: Master University Exam Proof Bank (Part IV)

🏛️ Step-by-Step Solved Proof: First-Order Unification with Multiple Variable Bindings

Find the Most General Unifier (MGU) θ for the following predicate expressions or show why unification fails:

1. $P(x, g(x), y)$ and $P(f(z), g(f(z)), h(w)) \implies \theta = \{\mathbf{x/f(z), y/h(w)}\}$.
2. $Q(a, x, f(g(y)))$ and $Q(z, f(z), f(u)) \implies z/a, x/f(a), u/g(y) \implies \theta = \{\mathbf{z/a, x/f(a), u/g(y)}\}$.
3. $R(x, x)$ and $R(y, f(y)) \implies$ Occur-check failure: $\mathbf{y}$ cannot unify with $\mathbf{f(y)}$!

Q18. Explain Automated Theorem Proving with Binary Decision Diagrams (BDDs) and Reduced Ordered BDDs (ROBDDs). (8 Marks)

A **Binary Decision Diagram (BDD)** is a rooted, directed acyclic graph representing a Boolean function $f(x_1, \ldots, x_n)$ where non-terminal nodes represent variables and outgoing dashed/solid edges represent 0 and 1 assignments.
ROBDD Canonical Property (Bryant 1986): For a fixed variable ordering, every Boolean function has a *strictly unique, canonical* ROBDD representation. Checking whether a formula is a tautology ($f \equiv 1$) or unsatisfiable ($f \equiv 0$) takes $O(1)$ time! Equivalence checking between two logic circuits $f \equiv g$ reduces to graph isomorphism!

MODULE 4 OF 5

Module 4: Classical Planning & Bayesian Networks

Topics 23 to 29 • STRIPS / PDDL, Graphplan Mutexes, Probability & Bayes Nets

Module 4 Table of Contents (Topics 23 to 29)

Topic 23 to 25: Classical Planning, STRIPS & Graphplan

In **Classical Planning**, states and actions are represented using explicit propositional or first-order fluents. Under the **STRIPS / PDDL language**:

STRIPS Action Representation:

$$\text{Action}(\mathbf{a}, \text{Precond} : \mathbf{P}, \text{Effect} : \text{Add}(\mathbf{A}) \wedge \text{Delete}(\mathbf{D}))$$

$$\text{Result}(\mathbf{s}, \mathbf{a}) = (\mathbf{s} \setminus \mathbf{D}) \cup \mathbf{A}$$

🏛️ STRIPS Blocks World Problem Formulation

```
Init(On(A, Table) ∧ On(B, Table) ∧ Clear(A) ∧ Clear(B) ∧ ArmEmpty)
Goal(On(A, B) ∧ On(B, Table))

Action(PickUp(x),
  PRECOND: Clear(x) ∧ On(x, Table) ∧ ArmEmpty
  EFFECT:  ¬On(x, Table) ∧ ¬Clear(x) ∧ ¬ArmEmpty ∧ Holding(x))

Action(PutDown(x),
  PRECOND: Holding(x)
  EFFECT:  On(x, Table) ∧ Clear(x) ∧ ArmEmpty ∧ ¬Holding(x))

Action(Stack(x, y),
  PRECOND: Holding(x) ∧ Clear(y)
  EFFECT:  On(x, y) ∧ Clear(x) ∧ ArmEmpty ∧ ¬Holding(x) ∧ ¬Clear(y))

Action(Unstack(x, y),
  PRECOND: On(x, y) ∧ Clear(x) ∧ ArmEmpty
  EFFECT:  Holding(x) ∧ Clear(y) ∧ ¬On(x, y) ∧ ¬Clear(x) ∧ ¬ArmEmpty)
```

🏛️ **Planning Graph Mutex (Mutual Exclusion) Conditions**

In Graphplan, two actions at level A_i are **MUTEX** if:

1. **Inconsistent Effects:** One action negates an effect of the other (A adds P, B deletes P).
2. **Interference:** One action deletes a precondition of the other (A deletes P, B requires P).
3. **Competing Needs:** Preconditions of A and B are mutex at literal level S_i.

Topic 26 to 29: Probability, Bayes' Rule & Bayesian Networks

Bayes' Rule & Conditional Independence:

$$\mathbf{P(Y \mid X) = \frac{P(X \mid Y)P(Y)}{P(X)} = \frac{P(X \mid Y)P(Y)}{\sum_y P(X \mid Y = y)P(Y = y)}}$$

$$\mathbf{P(X, Y \mid Z) = P(X \mid Z) \cdot P(Y \mid Z) \iff X \text{ and } Y \text{ conditionally independent given } Z}$$

🏛️ **Complete Step-by-Step Solved Problem: Earthquake-Burglary Bayesian Network Inference**

Given the classic Pearl Alarm network: B (Burglary), E (Earthquake), A (Alarm), J (JohnCalls), M (MaryCalls):

- $P(B = t) = 0.001, \quad P(E = t) = 0.002$
- $P(A = t \mid B = t, E = t) = 0.95,\ P(A = t \mid B = t, E = f) = 0.94,\ P(A = t \mid B = f, E = t) = 0.29,\ P(A = t \mid B = f, E = f) = 0.001$
- $P(J = t \mid A = t) = 0.90,\ P(J = t \mid A = f) = 0.05$
- $P(M = t \mid A = t) = 0.70,\ P(M = t \mid A = f) = 0.01$

Query: Calculate the exact joint probability that John calls and Mary calls, but there is NO burglary and NO earthquake, and the alarm did ring:

$$\mathbf{P(J = t, M = t, A = t, B = f, E = f) = P(B = f) \cdot P(E = f) \cdot P(A = t \mid B = f, E = f) \cdot P(J = t \mid A = t) \cdot P(M = t \mid A = t)}$$

$$= (0.999) \times (0.998) \times (0.001) \times (0.90) \times (0.70)$$

$$= \mathbf{0.997002 \times 0.001 \times 0.63 = 0.00062811 = 6.28 \times 10^{-4}}$$

Topic 29.2: Master University Examination Solved Question Bank (10 Solved Questions)

Q1. State and prove Bayes' Theorem from the Definition of Conditional Probability. (6 Marks)

By definition: $P(A \wedge B) = P(A \mid B)P(B)$ and $P(A \wedge B) = P(B \mid A)P(A)$. Equating the two expressions gives $P(A \mid B)P(B) = P(B \mid A)P(A)$. Dividing both sides by $P(B)$ yields: $\mathbf{P(A \mid B) = \frac{P(B|A)P(A)}{P(B)}}$.

Q2. Explain the difference between Progression Planning and Regression Planning. (8 Marks)

• **Progression Planning (Forward State-Space Search):** Starts from initial state S_0 and applies all applicable actions to generate successor states moving toward the goal. High branching factor on irrelevant actions.
• **Regression Planning (Backward Search):** Starts from the goal state G and applies inverted relevant actions backwards toward the initial state. Explores only actions that directly achieve a goal literal.

Q3. What is a Bayesian Network? How does it compress the Full Joint Probability Distribution? (8 Marks)

A Bayesian Network is a Directed Acyclic Graph (DAG) where nodes represent random variables and edges represent direct causal dependencies. For n binary variables, a Full Joint Distribution requires $2^n - 1$ parameters (exponential). A Bayes Net factors the joint distribution as: $\mathbf{P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{Parents}(X_i))}$. If each node has at most k parents, it requires only $n \cdot 2^k$ parameters (linear in n!).

Q4. State the d-Separation rules for Causal Chains, Forks, and Colliders. (8 Marks)

1. **Causal Chain ($X \to Z \to Y$):** Blocked/Independent given Z.
2. **Common Cause / Fork ($X \leftarrow Z \to Y$):** Blocked/Independent given Z.
3. **Common Effect / Collider ($X \to Z \leftarrow Y$):** Independent if neither Z nor any descendant of Z is observed; **Becomes DEPENDENT (Active)** if Z or its descendant is observed! (Explaining Away phenomenon).

Q5. Explain the Graphplan Algorithm and why Planning Graphs can be computed in polynomial time. (8 Marks)

Graphplan constructs a Planning Graph of alternating state levels S_i and action levels A_i connected by precondition, add, delete, and persistence edges. Since actions are monotonic (literals only accumulate, never deleted from the graph level), the graph levels level off in polynomial time $O(n(a+l))$. Solution extraction is performed by backward search on mutex-free paths.

Q6. Explain the Sussman Anomaly in Classical Planning. (6 Marks)

The Sussman Anomaly is a classic failure mode of non-interleaved goal planning where achieving goal subgoal 1 ($On(A,B)$) undoes the prerequisite for subgoal 2 ($On(B,C)$), and vice-versa. It proves that simple decomposition of conjunctive goals without plan-step interleaving is sub-optimal and incomplete.

Q7. Detail the Variable Elimination Algorithm for Exact Inference in Bayesian Networks. (8 Marks)

Variable Elimination computes marginal distributions by dynamic programming:
1. Express query as sum of products of conditional probability table factors.
2. Choose an elimination ordering of non-query, non-evidence hidden variables.
3. Multiply all factors containing the variable to eliminate and sum out the variable.
4. Pointwise multiply the remaining factor and normalize by α.

Q8. Explain Markov Chain Monte Carlo (MCMC) and Gibbs Sampling in Bayesian Networks. (8 Marks)

When exact inference is NP-hard on dense networks, Gibbs Sampling generates approximate samples by:
1. Initializing all non-evidence variables randomly.
2. In each iteration, picking one non-evidence variable X_i and resampling its value conditioned on its Markov Blanket $P(X_i \mid \text{MB}(X_i))$.
3. Counting the frequencies of sampled states after a burn-in period to estimate posterior probabilities.

Q9. Explain Partial-Order Planning (POP) and contrast it with Total-Order Planning. (8 Marks)

Total-Order Planning strictly sequences actions into a single linear chain. **Partial-Order Planning** operates in the space of plans, establishing ordering constraints ($A \prec B$) and causal links ($A \xrightarrow{p} B$) only where necessary to resolve causal threats (demotions/promotions), leaving independent sub-plans unordered to facilitate parallel execution.

Q10. Calculate $P(\text{Cavity} \mid \text{Toothache})$ given: $P(\text{Toothache} \mid \text{Cavity}) = 0.8$, $P(\text{Cavity}) = 0.1$, and $P(\text{Toothache}) = 0.2$. (6 Marks)

Using Bayes' Rule:

$$\mathbf{P}(\text{Cavity} \mid \text{Toothache}) = \frac{\mathbf{P}(\text{Toothache} \mid \text{Cavity})\mathbf{P}(\text{Cavity})}{\mathbf{P}(\text{Toothache})} = \frac{(\mathbf{0.8})(\mathbf{0.1})}{\mathbf{0.2}} = \frac{\mathbf{0.08}}{\mathbf{0.20}} = \mathbf{0.40} = \mathbf{40\%}$$

Topic 29.5: Advanced Probabilistic Reasoning, HMMs & Kalman Filters

When reasoning over time in dynamic stochastic environments, agents use temporal probabilistic models to maintain probability distributions over unobserved hidden state variables.

Temporal Model	Mathematical State Representation	Filtering & Inference Task
Hidden Markov Model (HMM)	Discrete state space $S_t \in \{s_1, \dots, s_N\}$, discrete or continuous observations O_t. Transition matrix $T_{ij} = P(S_t = s_j \mid S_{t-1} = s_i)$, Emission matrix $E_{ik} = P(O_t = o_k \mid S_t = s_i)$.	Forward Algorithm (Filtering $P(S_t \mid o_{1:t})$), Backward Algorithm (Smoothing), Viterbi Algorithm (Most likely hidden path).
Kalman Filter (LGM)	Continuous state space with linear Gaussian transitions: $\mathbf{x}_t = \mathbf{F}\mathbf{x}_{t-1} + \mathbf{w}_t, \mathbf{z}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t$ (where $\mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q}), \mathbf{v}_t \sim \mathcal{N}(0, \mathbf{R})$).	Exact recursive update of Gaussian mean μ_t and covariance matrix $\mathbf{\Sigma}_t$.
Particle Filter (Sequential Monte Carlo)	Arbitrary non-linear, non-Gaussian continuous state space represented by a cloud of N weighted particles $\{(\mathbf{x}_t^{(i)}, w_t^{(i)})\}_{i=1}^N$.	Importance sampling, weight update by measurement likelihood, and resampling to prevent particle degeneracy.

🏛️ Step-by-Step Solved Problem: The Viterbi Dynamic Programming Algorithm for HMMs

Consider a 2-state weather HMM with states $\{\text{Rainy } (R), \text{Sunny } (S)\}$ and umbrella observations $\{U, \neg U\}$:

- Initial: $P(R_0) = 0.5,\ P(S_0) = 0.5$.
- Transitions: $P(R \mid R) = 0.7,\ P(S \mid R) = 0.3;\ P(R \mid S) = 0.3,\ P(S \mid S) = 0.7$.
- Emissions: $P(U \mid R) = 0.9,\ P(\neg U \mid R) = 0.1;\ P(U \mid S) = 0.2,\ P(\neg U \mid S) = 0.8$.

Observations: Day 1: U, Day 2: U. Find most likely state sequence:

Day 1 ($t = 1, O_1 = U$):

$$V_1(R) = P(R_0) \cdot P(U \mid R) = (0.5)(0.9) = \mathbf{0.45}$$

$$V_1(S) = P(S_0) \cdot P(U \mid S) = (0.5)(0.2) = \mathbf{0.10}$$

Day 2 ($t = 2, O_2 = U$):

$$V_2(R) = \max(V_1(R)P(R \mid R), V_1(S)P(R \mid S)) \cdot P(U \mid R) = \max((0.45)(0.7), (0.10)(0.3)) \cdot 0.9 = \max(0.315, 0.030) \cdot 0.9 = \mathbf{0.2835}$$

$$V_2(S) = \max(V_1(R)P(S \mid R), V_1(S)P(S \mid S)) \cdot P(U \mid S) = \max((0.45)(0.3), (0.10)(0.7)) \cdot 0.2 = \max(0.135, 0.070) \cdot 0.2 = \mathbf{0.0270}$$

$$\text{Most Probable Path: Day 1: Rainy} \rightarrow \text{Day 2: Rainy} \quad (\text{Joint Prob } = \mathbf{0.2835})$$

Q11. Explain the difference between Policy Iteration and Value Iteration in Markov Decision Processes. (8 Marks)

• **Value Iteration:** Iteratively updates value function $U_{k+1}(s)$ using the Bellman Optimality operator over all actions until convergence ($\|U_{k+1} - U_k\| < \epsilon$), then extracts the policy in a single final step. Can be slow because utility values take many iterations to converge to exact decimal precision.
• **Policy Iteration:** Alternates between two discrete phases: (1) *Policy Evaluation:* Calculates exact state utilities U^{π_k} for the current fixed policy π_k by solving a system of linear equations $O(N^3)$, and (2) *Policy Improvement:* Greedily updates the policy $\pi_{k+1}(s) = \arg\max_a \sum_{s'} P(s' \mid s, a)U^{\pi_k}(s')$. It converges in significantly fewer iterations because the policy space is finite ($|\mathcal{A}|^{|\mathcal{S}|}$).

Q12. What is Q-Learning? Why is it termed an Off-Policy Model-Free Algorithm? (8 Marks)

• **Model-Free:** Does NOT require explicit transition probability matrices $P(s' \mid s, a)$ or reward functions $R(s, a, s')$; it learns directly from sampled experiential trajectories $\langle s, a, r, s' \rangle$.
• **Off-Policy:** The target policy being learned is the *greedy optimal policy* ($\max_{a'} Q(s', a')$), which is completely decoupled from the *behavioral policy* (e.g. ϵ-greedy exploration) used to generate actions in the environment.

Topic 29.6: Advanced Decision Networks & Value of Information (VOI)

Value of Perfect Information (VPI / VOI): The expected value of information regarding an unobserved random variable E_j is the difference between the expected utility with E_j known versus without knowing E_j:

$$\text{VPI}(\mathbf{E_j}) = \left(\sum_{\mathbf{k}} \mathbf{P}(\mathbf{E_j} = \mathbf{e_{jk}}) \max_{\mathbf{a}} \mathbb{E}[\mathbf{U}(\mathbf{a} \mid \mathbf{e_{jk}})] \right) - \max_{\mathbf{a}} \mathbb{E}[\mathbf{U}(\mathbf{a})]$$

Properties of VPI: 1. Non-negative: $\text{VPI}(E_j) \geq 0$ (Information never reduces expected utility).
2. Non-additive: $\text{VPI}(E_j, E_k) \neq \text{VPI}(E_j) + \text{VPI}(E_k)$.
3. Order-independent: $\text{VPI}(E_j, E_k) = \text{VPI}(E_j) + \text{VPI}(E_k \mid E_j) = \text{VPI}(E_k) + \text{VPI}(E_j \mid E_k)$.

🏛️ Step-by-Step Solved Problem: Value of Information in Oil Drilling

An oil company is deciding whether to drill an offshore site (a_1) or sell the lease for $\$3\text{M}$ (a_2). Prior probability of oil is $P(\text{Oil}) = 0.5$. If oil is present, profit is $\$10\text{M}$; if dry, loss is $-\$4\text{M}$.

- Expected utility of drilling (a_1): $\mathbb{E}[U(a_1)] = 0.5(10) + 0.5(-4) = 5 - 2 = \mathbf{\$3}\text{M}$.
- Expected utility of selling (a_2): $\mathbb{E}[U(a_2)] = \mathbf{\$3}\text{M}$. Base optimal utility = $\$3\text{M}$.

A seismic survey (S) provides perfect information about whether oil is present:

- If seismic says Oil ($P = 0.5$): Drill (a_1) $\implies$ $\text{Utility} = \$10\text{M}$.
- If seismic says Dry ($P = 0.5$): Sell (a_2) $\implies$ $\text{Utility} = \$3\text{M}$.
- Expected utility with seismic test: $0.5(10) + 0.5(3) = 5 + 1.5 = \mathbf{\$6.5}\text{M}$.

$$\text{VPI}(\text{Seismic}) = \mathbf{\$6.5}\text{M} - \mathbf{\$3}\text{M} = \mathbf{\$3.5}\text{M}$$

Decision: The company should pay up to $\$3.5\text{M}$ for the seismic survey test!

Topic 29.7: Partially Observable MDPs (POMDPs) & Dynamic Decision Networks

When state transitions are stochastic and the state is only **partially observable** through noisy sensors, the agent operates in a **POMDP (Partially Observable Markov Decision Process)**:

The 7-Tuple POMDP Formalism:

$$\mathcal{P} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}), \mathcal{R}(\mathbf{s}, \mathbf{a}), \mathbf{\Omega}, \mathcal{O}(\mathbf{o} \mid \mathbf{s}', \mathbf{a}), \gamma \rangle$$

Where Ω is the set of observations, and $\mathcal{O}(o \mid s', a)$ is the observation emission probability. The agent maintains a **Belief State** $b(s)$ (a continuous probability distribution over all states $\sum_s b(s) = 1$).

🏛️ The Exact Belief State Update Equation (Bayesian Filtering)

After executing action a and perceiving observation o, the updated belief $b'(s')$ is computed via Bayes' Rule:

$$\mathbf{b}'(\mathbf{s}') = \frac{\mathcal{O}(\mathbf{o} \mid \mathbf{s}', \mathbf{a}) \sum_{\mathbf{s} \in \mathcal{S}} \mathcal{P}(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}) \mathbf{b}(\mathbf{s})}{\mathbf{P}(\mathbf{o} \mid \mathbf{a}, \mathbf{b})} = \alpha \cdot \mathcal{O}(\mathbf{o} \mid \mathbf{s}', \mathbf{a}) \sum_{\mathbf{s} \in \mathcal{S}} \mathcal{P}(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}) \mathbf{b}(\mathbf{s})$$

Interpretation: The continuous belief space $[0, 1]^{|\mathcal{S}|}$ converts a partially observable problem into a continuous, fully observable MDP over belief states!

🏛️ Step-by-Step Solved Problem: POMDP Belief State Update Calculation

A robot is in a 2-room corridor (S_1, S_2). Prior belief $b(S_1) = 0.5, b(S_2) = 0.5$. Action $a = \text{MoveRight}$:

- Transitions: $P(S_2 \mid S_1, \text{MoveRight}) = 0.8, P(S_1 \mid S_1, \text{MoveRight}) = 0.2;\ P(S_2 \mid S_2, \text{MoveRight}) = 0.9, P(S_1 \mid S_2, \text{MoveRight}) = 0.1$.
- Light sensor reading: $o = \text{Bright}$. Emissions: $P(\text{Bright} \mid S_1) = 0.1, P(\text{Bright} \mid S_2) = 0.8$.

Calculate updated belief $b'(S_1)$ and $b'(S_2)$:

$$\text{Predicted } P(S_1) = (0.2)(0.5) + (0.1)(0.5) = 0.10 + 0.05 = \mathbf{0.15}$$

$$\text{Predicted } P(S_2) = (0.8)(0.5) + (0.9)(0.5) = 0.40 + 0.45 = \mathbf{0.85}$$

$$b'(S_1) \propto P(\text{Bright} \mid S_1) \cdot 0.15 = (0.1)(0.15) = 0.015$$

$$b'(S_2) \propto P(\text{Bright} \mid S_2) \cdot 0.85 = (0.8)(0.85) = 0.680$$

$$\text{Normalization Constant } \alpha = \frac{1}{0.015 + 0.680} = \frac{1}{0.695} \approx 1.4388$$

$$\mathbf{b'(S_1) = 0.015 \times 1.4388 = \mathbf{0.0216 = 2.16\%} \qquad b'(S_2) = 0.680 \times 1.4388 = \mathbf{0.9784 = 97.84\%}}$$

Topic 29.8: Complete Step-by-Step Solved Problem Bank (Part III)

🏛️ Solved Numerical 3: Bayesian Network Exact Joint Probability Calculation

Consider a 4-node Bayesian Network with DAG structure $A \to B \to D$ and $A \to C \to D$:

- $P(A = t) = 0.4$.
- $P(B = t \mid A = t) = 0.8,\ P(B = t \mid A = f) = 0.2$.
- $P(C = t \mid A = t) = 0.7,\ P(C = t \mid A = f) = 0.1$.
- $P(D = t \mid B = t, C = t) = 0.9,\ P(D = t \mid B = t, C = f) = 0.6,\ P(D = t \mid B = f, C = t) = 0.5,\ P(D = t \mid B = f, C = f) = 0.1$.

Calculate $P(A = t, B = t, C = f, D = t)$:

$$\mathbf{P(A = t, B = t, C = f, D = t) = P(A = t) \cdot P(B = t \mid A = t) \cdot P(C = f \mid A = t) \cdot P(D = t \mid B = t, C = f)}$$

$$= (0.4) \times (0.8) \times (1 - 0.7) \times (0.6)$$

$$= (0.4) \times (0.8) \times (0.3) \times (0.6) = 0.32 \times 0.18 = \mathbf{0.0576 = 5.76\%}$$

🏛️ Step-by-Step Solved Problem: STRIPS Shakey the Robot World

Formulate the STRIPS operators for a mobile robot moving between rooms and pushing boxes:

```
Action(Go(x, y),
  PRECOND: At(Shakey, x) ∧ In(x, r) ∧ In(y, r)
  EFFECT:  ¬At(Shakey, x) ∧ At(Shakey, y))

Action(Push(b, x, y),
  PRECOND: At(Shakey, x) ∧ At(b, x) ∧ In(x, r) ∧ In(y, r) ∧ Box(b)
  EFFECT:  ¬At(Shakey, x) ∧ ¬At(b, x) ∧ At(Shakey, y) ∧ At(b, y))

Action(ClimbOn(b),
  PRECOND: At(Shakey, x) ∧ At(b, x) ∧ On(Shakey, Floor) ∧ Box(b)
  EFFECT:  ¬On(Shakey, Floor) ∧ On(Shakey, b))

Action(TurnOnLight(s),
  PRECOND: At(Shakey, x) ∧ At(s, x) ∧ On(Shakey, b) ∧ Box(b) ∧ Switch(s)
  EFFECT:  LightOn(s))
```

Q13. Explain Hierarchical Task Networks (HTN) Planning. (8 Marks)

HTN Planning extends classical planning by decomposing high-level compound abstract tasks (e.g. `TravelTo(NewYork)`) into smaller sub-tasks using **Methods** until only primitive actions remain (e.g., `BuyTicket`, `DriveToAirport`, `Fly`). HTN plans are dramatically faster to compute than STRIPS because domain-specific expert knowledge restricts the search space to realistic human-like decompositions rather than searching through arbitrary combinations of primitive actions!

Topic 29.9: Complete Partial-Order Planning (POP) Algorithm & Flaw Repair

🏛️ Step-by-Step Solved Problem: Partial-Order Planning (POP) Algorithm Walkthrough

Consider the classic **"Put on Shoes and Socks"** planning domain:

- Actions: $\text{RightSock}, \text{RightShoe}, \text{LeftSock}, \text{LeftShoe}$.
- Preconditions & Effects:
  - RightShoe: Precondition RightSockOn; Effect RightShoeOn.
  - LeftShoe: Precondition LeftSockOn; Effect LeftShoeOn.
- Goal: $\text{RightShoeOn} \wedge \text{LeftShoeOn}$.

POP Execution Steps:

1. Start with initial dummy plan: $\text{Start} \prec \text{Finish}$. Open preconditions: $\text{RightShoeOn}, \text{LeftShoeOn}$.
2. Achieve RightShoeOn by adding step RightShoe with causal link $\text{RightShoe} \xrightarrow{\text{RightShoeOn}} \text{Finish}$.
3. Achieve LeftShoeOn by adding step LeftShoe with causal link $\text{LeftShoe} \xrightarrow{\text{LeftShoeOn}} \text{Finish}$.
4. Resolve open precondition RightSockOn of RightShoe by adding RightSock with causal link $\text{RightSock} \xrightarrow{\text{RightSockOn}} \text{RightShoe}$.
5. Resolve open precondition LeftSockOn of LeftShoe by adding LeftSock with causal link $\text{LeftSock} \xrightarrow{\text{LeftSockOn}} \text{LeftShoe}$.
6. **Threat Checking:** No action deletes any established causal link ($\text{Threats} = \emptyset$).

$$\text{Final Partial Order Plan: } (\text{RightSock} \prec \text{RightShoe}) \wedge (\text{LeftSock} \prec \text{LeftShoe})$$

Power of POP: The left-foot and right-foot operations remain completely unordered with respect to each other, allowing parallel execution on a dual-arm robot!

🏛️ Complete Step-by-Step MCMC Gibbs Sampling Trace on Bayesian Networks

Given 3 variables C (Cloudy), R (Rain), W (WetGrass) with query $P(C \mid W = \text{true})$:

- Evidence: $W = \text{true}$. Hidden non-evidence variables: C, R.
- **Step 0:** Randomly initialize hidden variables: $C_0 = \text{true},\ R_0 = \text{false}$. State = $(\text{true}, \text{false}, \text{true})$.
- **Iteration 1 (Sample C):** Compute $P(C \mid R = \text{false}, W = \text{true}) = \alpha P(C) P(R = \text{false} \mid C) P(W = \text{true} \mid C, R = \text{false})$. Suppose distribution evaluates to $[0.35, 0.65]$. Sample random $u = 0.42 > 0.35 \implies C_1 = \text{false}$.
- **Iteration 2 (Sample R):** Compute $P(R \mid C = \text{false}, W = \text{true})$. Suppose evaluates to $[0.82, 0.18]$. Sample random $u = 0.10 < 0.82 \implies R_1 = \text{true}$.
- **Iteration 3 to N:** Repeat sampling. Tally the number of iterations where $C = \text{true}$ versus $C = \text{false}$ to compute the empirical posterior probability!

Topic 29.10: Master University Exam Problem Bank (Part IV)

🏛️ Solved Numerical 4: Exact Inference by Enumeration in 5-Node Bayesian Network

Given DAG $A \to B \to C$, with conditional probability tables:

- $P(A = t) = 0.3$.
- $P(B = t \mid A = t) = 0.9,\ P(B = t \mid A = f) = 0.1$.
- $P(C = t \mid B = t) = 0.8,\ P(C = t \mid B = f) = 0.2$.

Query: Compute posterior probability $P(A = t \mid C = t)$:

$$P(A = t, C = t) = P(A = t) \sum_{b} P(b \mid A = t) P(C = t \mid b) = 0.3[(0.9)(0.8) + (0.1)(0.2)] = 0.3[0.72 + 0.02] = 0.3(0.74) = \mathbf{0.222}$$

$$P(A = f, C = t) = P(A = f) \sum_{b} P(b \mid A = f) P(C = t \mid b) = 0.7[(0.1)(0.8) + (0.9)(0.2)] = 0.7[0.08 + 0.18] = 0.7(0.26) = \mathbf{0.182}$$

$$\mathbf{P(A = t \mid C = t) = \frac{0.222}{0.222 + 0.182} = \frac{0.222}{0.404} = 0.5495 = 54.95\%}$$

🏛️ Step-by-Step Solved Problem: STRIPS Tire Changing Domain

```
Init(Tire(Flat) ∧ Tire(Spare) ∧ At(Flat, Axle) ∧ At(Spare, Trunk))
Goal(At(Spare, Axle) ∧ At(Flat, Trunk))

Action(Remove(t, loc),
  PRECOND: At(t, loc)
  EFFECT:  ¬At(t, loc) ∧ Holding(t))

Action(PutOn(t, Axle),
  PRECOND: Holding(t) ∧ ¬At(Flat, Axle) ∧ ¬At(Spare, Axle)
  EFFECT:  At(t, Axle) ∧ ¬Holding(t))

Action(PutIn(t, Trunk),
  PRECOND: Holding(t)
  EFFECT:  At(t, Trunk) ∧ ¬Holding(t))
```

Q14. Explain Real-Time Heuristic Search (RTA* and LRTA*). (8 Marks)

In dynamic physical environments where planning time is strictly bounded, **Learning Real-Time A^* (LRTA*)** executes action moves within fixed time bounds by searching forward only a few steps, evaluating leaf nodes with heuristic $h(n)$, choosing the best immediate action, and updating its heuristic table at the visited state s: $\mathbf{h(s) \leftarrow \min_a [c(s, a, s') + h(s')]}$. Over repeated trials, LRTA* is guaranteed to converge to the optimal path!

Topic 29.11: Formal Decision Theory, Axioms of Utility & Influence Diagrams

Decision Theory unifies probability theory with utility theory: $\text{Decision Theory} = \text{Probability Theory} + \text{Utility Theory}$. An agent represents preferences between lotteries using scalar utility functions.

Axiom of Rationality (Von Neumann-Morgenstern)	Mathematical Formulation	Implication for Rational Agents
Orderability (Completeness)	$(A \succ B) \vee (B \succ A) \vee (A \sim B)$	An agent cannot avoid making a choice between any two states or lotteries.
Transitivity	$(A \succ B) \wedge (B \succ C) \implies (A \succ C)$	Prevents cyclical preferences (intransitive agents become "money pumps" exploited by opponents).
Continuity	$A \succ B \succ C \implies \exists p \in [0,1] \text{ s.t. } [p, A; (1-p), C] \sim B$	There is always a lottery between best and worst outcomes equivalent to an intermediate outcome.
Substitutability (Independence)	$A \sim B \implies [p, A; (1-p), C] \sim [p, B; (1-p), C]$	Indifference between two outcomes is preserved when embedded in larger compound lotteries.
Monotonicity	$A \succ B \wedge (p > q) \implies [p, A; (1-p), B] \succ [q, A; (1-q), B]$	Agents strictly prefer lotteries with higher probabilities of receiving the more desirable prize.

🏛️ Step-by-Step Solved Problem: Influence Diagram Evaluation Algorithm

An **Influence Diagram (Decision Network)** augments Bayesian Networks with *Decision Nodes* (rectangles) and *Utility Nodes* (diamonds):

1. **Evaluation Algorithm:**
   - For each possible assignment to the decision node $D = d_i$:
   - Set evidence $D = d_i$ in the network.
   - Calculate posterior probabilities $P(X \mid d_i)$ for parents X of the utility node U using Variable Elimination.
   - Compute expected utility: $\mathbb{E}[U \mid d_i] = \sum_x P(X = x \mid d_i) U(x, d_i)$.
2. **Optimal Policy:** Choose action $d^* = \arg\max_{d_i} \mathbb{E}[U \mid d_i]$!

🏛️ The St. Petersburg Paradox & Risk Neutrality vs. Risk Aversion

A fair coin is tossed until heads appears on flip n. The player receives prize $\$2^n$. What is the fair price to play?

- **Expected Monetary Value (EMV):** $\mathbb{E}[\text{Money}] = \sum_{n=1}^{\infty} \left(\frac{1}{2}\right)^n (2^n) = \sum_{n=1}^{\infty} 1 = \mathbf{1} + \mathbf{1} + \mathbf{1} + \cdots = \boldsymbol{\infty}$.
- Yet real humans will only pay $\sim \$10$ to $\$25$ to play this game!
- **Bernoulli's Resolution (1738):** Humans maximize *Expected Utility* $U(w) = \ln(w)$ (concave logarithmic utility function), NOT raw monetary payout!

$$\mathbb{E}[U] = \sum_{n=1}^{\infty} \left(\frac{1}{2}\right)^n \ln(2^n) = \ln(2) \sum_{n=1}^{\infty} \frac{n}{2^n} = \ln(2) \times 2 = \mathbf{2}\ln(\mathbf{2}) \approx \mathbf{1.386} \text{ utils}$$

$$U^{-1}(1.386) = e^{1.386} = \mathbf{\$4.00} \text{ (Certainty Equivalent)}$$

🏛️ Step-by-Step Solved Problem: Policy Iteration Dynamic Programming Trace on 3-State Chain

Consider a 3-state chain MDP with states $\{S_1, S_2, S_3\}$, actions $\{\text{Left}, \text{Right}\}$, discount $\gamma = 0.8$, and terminal absorbing state S_3 with reward $R(S_3) = +10$. Other step rewards are 0.

1. Initial Policy π_0: Always choose Right ($\pi_0(S_1) = \text{Right}, \pi_0(S_2) = \text{Right}$).

2. Policy Evaluation: Solve linear system $U^{\pi_0}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_0(s)) U^{\pi_0}(s')$:

$$U(S_2) = 0 + 0.8 U(S_3) = 0.8(10) = \mathbf{8.0}$$

$$U(S_1) = 0 + 0.8 U(S_2) = 0.8(8.0) = \mathbf{6.4}$$

3. Policy Improvement: Calculate $Q(s, a)$ for alternative action Left:

$$Q(S_1, \text{Left}) = 0 + 0.8 U(S_1) = 0.8(6.4) = 5.12 < 6.4 \implies \text{Keep Right}$$

$$Q(S_2, \text{Left}) = 0 + 0.8 U(S_1) = 0.8(6.4) = 5.12 < 8.0 \implies \text{Keep Right}$$

$$\text{Policy Converged: } \pi^*(\mathbf{S_1}) = \text{Right},\ \pi^*(\mathbf{S_2}) = \text{Right} \quad (\text{Optimal Policy Identified in 1 Iteration!})$$

MODULE 5 OF 5

Module 5: Machine Learning & Neural Networks

Topics 30 to 38 • Decision Trees ID3, Perceptrons & Backpropagation Math

Module 5 Table of Contents (Topics 30 to 38)

Topic 30 to 33: Decision Tree Induction & Information Gain Mathematics

In **Decision Tree Learning**, candidate attributes are recursively evaluated at each split to maximize the purity of child nodes:

Shannon Entropy & Information Gain Formulas:

$$\mathbf{H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)}$$

$$\mathbf{\text{Gain}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)}$$

$$\mathbf{\text{Gini Index: } \text{Gini}(S) = 1 - \sum_{i=1}^{c} p_i^2}$$

Step-by-Step Solved Problem: ID3 Decision Tree Root Attribute Selection

A dataset has 14 instances (9 Yes, 5 No). Initial parent entropy is:

$$H(S) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = -(0.643)(-0.637) - (0.357)(-1.485) = \mathbf{0.940} \text{ bits}$$

Evaluate Attribute **Windy** ($\text{Weak: } 6 \text{ Yes}, 2 \text{ No};\ \text{Strong: } 3 \text{ Yes}, 3 \text{ No}$):

$$H(S_{\text{Weak}}) = -\frac{6}{8}\log_2\left(\frac{6}{8}\right) - \frac{2}{8}\log_2\left(\frac{2}{8}\right) = \mathbf{0.811} \text{ bits}$$

$$H(S_{\text{Strong}}) = -\frac{3}{6}\log_2\left(\frac{3}{6}\right) - \frac{3}{6}\log_2\left(\frac{3}{6}\right) = \mathbf{1.000} \text{ bits}$$

$$\text{Gain}(S, \text{Windy}) = 0.940 - \left[\frac{8}{14}(0.811) + \frac{6}{14}(1.000)\right] = 0.940 - (0.463 + 0.429) = \mathbf{0.048} \text{ bits}$$

Topic 34 to 38: Neural Networks, Backpropagation & Non-Linear Activations

🏛️ Why Single-Layer Perceptrons Cannot Solve XOR (Minsky & Papert 1969)

A single perceptron $y = \text{step}(w_1 x_1 + w_2 x_2 - \theta)$ represents a single straight hyper-plane linear decision boundary. The XOR function requires:

$$\text{For } (0,0) \to 0: \; 0 < \theta \implies \theta > 0$$

$$\text{For } (1,0) \to 1: \; w_1 \geq \theta$$

$$\text{For } (0,1) \to 1: \; w_2 \geq \theta$$

$$\text{For } (1,1) \to 0: \; w_1 + w_2 < \theta$$

$$\text{Adding } (1,0) \text{ and } (0,1) \implies w_1 + w_2 \geq 2\theta > \theta \implies \text{CONTRADICTION! (Non-linearly separable)}$$

Complete Mathematical Derivation of Backpropagation Gradient: Let squared loss be $E = \frac{1}{2}\sum_k (y_k - \hat{y}_k)^2$. Using the Multivariate Chain Rule for output layer weight w_{jk}:

$$\frac{\partial \mathbf{E}}{\partial \mathbf{w_{jk}}} = \frac{\partial \mathbf{E}}{\partial \hat{\mathbf{y}}_\mathbf{k}} \cdot \frac{\partial \hat{\mathbf{y}}_\mathbf{k}}{\partial \mathbf{z_k}} \cdot \frac{\partial \mathbf{z_k}}{\partial \mathbf{w_{jk}}} = -(\mathbf{y_k} - \hat{\mathbf{y}}_\mathbf{k}) \cdot \sigma'(\mathbf{z_k}) \cdot \mathbf{a_j} = -\delta_\mathbf{k} \cdot \mathbf{a_j}$$

$$\text{Weight Update: } \mathbf{w_{jk}} \leftarrow \mathbf{w_{jk}} - \eta \frac{\partial \mathbf{E}}{\partial \mathbf{w_{jk}}} = \mathbf{w_{jk}} + \eta \cdot \delta_\mathbf{k} \cdot \mathbf{a_j}$$

For hidden layer weight w_{ij}:

$$\delta_\mathbf{j} = \sigma'(\mathbf{z_j}) \sum_\mathbf{k} \delta_\mathbf{k} \mathbf{w_{jk}} \implies \mathbf{w_{ij}} \leftarrow \mathbf{w_{ij}} + \eta \cdot \delta_\mathbf{j} \cdot \mathbf{a_i}$$

Activation Function	Mathematical Formula	Output Range	Derivative $\sigma'(z)$
Sigmoid (Logistic)	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	$\sigma(z)(1-\sigma(z))$
Hyperbolic Tangent (Tanh)	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$ (Zero-centered)	$1 - \tanh^2(z)$
ReLU (Rectified Linear)	$f(z) = \max(0, z)$	$[0, \infty)$	1 if $z > 0$ else 0
Softmax	$\sigma(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0,1)$ with $\sum p_i = 1$	$p_i(\delta_{ij} - p_j)$

Topic 38.2: Master University Examination Solved Question Bank (10 Solved Questions)

Q1. State the Perceptron Convergence Theorem. What happens if the training data is not linearly separable? (8 Marks)

The **Perceptron Convergence Theorem** (Rosenblatt 1962) proves that if a dataset is linearly separable (there exists a hyperplane with margin $\gamma > 0$), the perceptron learning rule is guaranteed to find a separating hyperplane in a finite number of weight updates $k \leq \left(\frac{R}{\gamma}\right)^2$ (where $R = \max \|x\|$). If the dataset is *non-linearly separable*, the perceptron weights oscillate indefinitely and never converge.

Q2. Complete Step-by-Step Backpropagation Numerical Walkthrough with explicit calculations. (12 Marks)

Consider a network: $x_1 = 0.05, x_2 = 0.10$, target $y = 0.01$, $\eta = 0.5$. Hidden weights: $w_1 = 0.15, w_2 = 0.20, w_3 = 0.25, w_4 = 0.30$, bias $b_1 = 0.35$. Output weights: $w_5 = 0.40, w_6 = 0.45$, bias $b_2 = 0.60$.
1. Forward Pass: $z_{h1} = (0.05)(0.15) + (0.10)(0.20) + 0.35 = 0.3775 \implies a_{h1} = \sigma(0.3775) = 0.59327$. $z_{h2} = (0.05)(0.25) + (0.10)(0.30) + 0.35 = 0.3925 \implies a_{h2} = \sigma(0.3925) = 0.59688$. Output: $z_{o1} = (0.59327)(0.40) + (0.59688)(0.45) + 0.60 = 1.1059 \implies a_{o1} = \sigma(1.1059) = \mathbf{0.75137}$. Error $E = \frac{1}{2}(0.01 - 0.75137)^2 = \mathbf{0.2748}$.
2. Backward Pass: $\delta_{o1} = (y - a_{o1})a_{o1}(1 - a_{o1}) = (0.01 - 0.75137)(0.75137)(0.24863) = \mathbf{-0.1385}$. Weight gradient: $\frac{\partial E}{\partial w_5} = -\delta_{o1} a_{h1} = -(-0.1385)(0.59327) = +\mathbf{0.08217}$. New weight: $\mathbf{w_5^{new}} = \mathbf{0.40} - \mathbf{0.5}(\mathbf{0.08217}) = \mathbf{0.3589}$.

Q3. Contrast ID3, C4.5, and CART Decision Tree Algorithms. (8 Marks)

• **ID3 (Quinlan):** Uses Information Gain (Entropy). Restricted to discrete categorical attributes; biased toward attributes with many values.
• **C4.5 (Quinlan):** Uses *Gain Ratio* ($\text{Gain}/\text{SplitInfo}$) to eliminate multi-value bias. Handles continuous attributes (dynamic thresholds) and missing values. Employs post-pruning.
• **CART (Breiman):** Uses *Gini Impurity*. Builds strictly binary decision trees and supports both classification and regression.

Q4. Explain the Bias-Variance Trade-off in Machine Learning with an error decomposition derivation. (8 Marks)

Expected prediction error on unseen data decomposes into: $\mathbb{E}[(\mathbf{y} - \hat{\mathbf{f}}(\mathbf{x}))^2] = \text{Bias}^2(\hat{\mathbf{f}}(\mathbf{x})) + \text{Var}(\hat{\mathbf{f}}(\mathbf{x})) + \sigma^2$ (where σ^2 is irreducible noise).
• **High Bias (Underfitting):** Model is overly simplistic and fails to capture underlying true relationships (e.g. linear model on quadratic data).
• **High Variance (Overfitting):** Model is overly complex and fits random training noise, failing to generalize to test data.

Q5. Explain the Vanishing Gradient Problem in Deep Neural Networks and how ReLU resolves it. (8 Marks)

For Sigmoid activation $\sigma(z) = \frac{1}{1+e^{-z}}$, the derivative is $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$. In a network with L layers, backpropagation multiplies gradients across layers: $\prod_{l=1}^{L} \sigma'(z_l) \leq (0.25)^L$. For $L = 10$, $(0.25)^{10} \approx 10^{-6}$, causing earlier layer weights to freeze.
ReLU Resolution: $f(z) = \max(0, z) \implies f'(z) = 1$ for all $z > 0$. Gradients pass through unchanged without diminishing!

Q6. Explain Reinforcement Learning (RL), the Markov Decision Process (MDP), and the Bellman Equation. (10 Marks)

In RL, an agent learns optimal behavior through trial-and-error interactions with an environment by receiving scalar rewards $R(s, a, s')$. An MDP is defined by $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$.
The Bellman Optimality Equation:

$$\mathbf{V}^*(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \sum_{\mathbf{s}'} \mathbf{P}(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}) \left[\mathbf{R}(\mathbf{s}, \mathbf{a}, \mathbf{s}') + \gamma \mathbf{V}^*(\mathbf{s}') \right]$$

$$\mathbf{Q}^*(\mathbf{s}, \mathbf{a}) = \sum_{\mathbf{s}'} \mathbf{P}(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}) \left[\mathbf{R}(\mathbf{s}, \mathbf{a}, \mathbf{s}') + \gamma \max_{\mathbf{a}'} \mathbf{Q}^*(\mathbf{s}', \mathbf{a}') \right]$$

Q7. Explain Q-Learning (Off-Policy Temporal Difference Control). (8 Marks)

Q-learning directly learns the optimal action-value function $Q^*(s, a)$ independent of the agent's behavioral exploration policy:

$$\mathbf{Q}(\mathbf{s}, \mathbf{a}) \leftarrow \mathbf{Q}(\mathbf{s}, \mathbf{a}) + \alpha \left[\mathbf{r} + \gamma \max_{\mathbf{a}'} \mathbf{Q}(\mathbf{s}', \mathbf{a}') - \mathbf{Q}(\mathbf{s}, \mathbf{a}) \right]$$

Where $\alpha \in (0, 1]$ is the learning rate, $\gamma \in [0, 1)$ is the discount factor, and $[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ is the TD Error.

Q8. Explain Ensemble Learning: Bagging (Random Forests) vs. Boosting (AdaBoost/XGBoost). (8 Marks)

• **Bagging (Bootstrap Aggregating):** Trains multiple independent base models (e.g., deep decision trees) in parallel on bootstrap sample subsets with replacement and averages predictions. Primary goal: *Reduces Variance*.
• **Boosting:** Trains base learners (e.g. shallow decision stumps) sequentially, where each new learner focuses on correcting the errors (higher instance weights) of previous models. Primary goal: *Reduces Bias*.

Q9. Explain Support Vector Machines (SVM), Maximum Margin Hyperplane, and the Kernel Trick. (8 Marks)

SVM finds the separating hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$ that maximizes the geometric margin $\frac{2}{\|\mathbf{w}\|}$ between support vectors (quadratic optimization). The **Kernel Trick** $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ (e.g. RBF Gaussian $e^{-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2}$, Polynomial) implicitly projects data into infinite-dimensional Hilbert space to achieve linear separability without explicitly computing high-dimensional coordinates.

Q10. Explain k-Nearest Neighbors (k-NN) Classification and Distance Metrics. (6 Marks)

k-NN is a non-parametric, lazy instance-based learning algorithm that stores all training samples. Given a test query $\mathbf{x}_q$, it calculates distances (Euclidean $d = \sqrt{\sum (x_i - y_i)^2}$, Manhattan $d = \sum |x_i - y_i|$) to all points, identifies the k nearest neighbors, and assigns the majority class label (or distance-weighted vote).

Topic 38.5: Modern Deep Generative Models, Transformers & LLM Architectures

Modern Artificial Intelligence is driven by large-scale deep learning architectures capable of representation learning across raw multimodal sensory data.

Generative Model	Training Objective & Loss Function	Key AI Applications
Variational Autoencoders (VAE)	Maximizes the Evidence Lower Bound (ELBO): $$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(z\|x)}[\log p_\theta(x \mid z)] - D_{\text{KL}}(q_\phi(z \mid x) \parallel p(z))$$	Latent space interpolation, image generation, anomaly detection.
Generative Adversarial Networks (GAN)	Minimax Two-Player Game: $$\min_G \max_D V(D,G) = \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))]$$	Photorealistic image synthesis, deepfakes, super-resolution.
Diffusion Models (DDPM)	Forward Markov chain adds Gaussian noise ($q(x_t \mid x_{t-1})$); reverse neural network $\epsilon_\theta(x_t, t)$ learns to denoise step-by-step.	State-of-the-art text-to-image (Stable Diffusion, Midjourney, DALL-E 3).
Autoregressive Transformers (GPT)	Causal Language Modeling (Next-token cross-entropy loss): $$\mathcal{L}_{\text{CLM}} = -\sum_{t=1}^{T} \log P(w_t \mid w_1, \ldots, w_{t-1}; \theta)$$	Large Language Models, code generation, reasoning agents.

🏛️ The Transformer Scaled Dot-Product Self-Attention Mathematical Formulation

Given an input sequence of token embeddings $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{model}}}$, linear projection matrices produce Queries $\mathbf{Q}$, Keys $\mathbf{K}$, and Values $\mathbf{V}$:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_\mathbf{Q}, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_\mathbf{K}, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_\mathbf{V} \qquad (\mathbf{W}_\mathbf{Q}, \mathbf{W}_\mathbf{K} \in \mathbb{R}^{\mathbf{d}_{\text{model}} \times \mathbf{d}_\mathbf{k}},\ \mathbf{W}_\mathbf{V} \in \mathbb{R}^{\mathbf{d}_{\text{model}} \times \mathbf{d}_\mathbf{v}})$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\mathbf{T}}{\sqrt{\mathbf{d}_\mathbf{k}}}\right)\mathbf{V}$$

Why Divide by $\sqrt{d_k}$? For large dimensional vectors d_k, the dot products grow large in magnitude, pushing the softmax function into regions with extremely small gradients (vanishing gradients). Scaling by $\frac{1}{\sqrt{d_k}}$ preserves unit variance and stabilizes gradient flow!

Q11. Explain Convolutional Neural Networks (CNNs) and the significance of Parameter Sharing and Equivariance. (8 Marks)

• **Parameter Sharing:** The same kernel filter weights are applied across every spatial receptive field of the input tensor, drastically reducing total learnable weights compared to fully-connected layers.
• **Translational Equivariance:** If an input object shifts by (dx, dy), the output feature map activates at the corresponding shifted location ($(f * g)(x - dx) = f(x - dx) * g$). This allows CNNs to recognize visual features anywhere in an image!

Q12. Explain Reinforcement Learning from Human Feedback (RLHF) for Large Language Models. (8 Marks)

RLHF aligns raw pre-trained LLMs with human values (Helpful, Honest, Harmless) via a 3-stage pipeline:
1. **Supervised Fine-Tuning (SFT):** Fine-tune base LLM on high-quality human prompt-response demonstrations.
2. **Reward Model (RM) Training:** Prompt the SFT model to generate multiple candidate outputs for a prompt; human evaluators rank them. Train a scalar reward model $r_\theta(x, y)$ using Bradley-Terry preference loss: $\mathcal{L} = -\mathbb{E}[\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))]$.
3. **PPO Policy Optimization:** Optimize the LLM policy using Proximal Policy Optimization (PPO) to maximize reward $r_\theta(x, y)$ while penalizing KL-divergence $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{SFT}})$ from the reference model to prevent reward hacking.

Topic 38.6: Advanced Statistical Learning Theory & Optimization Algorithms

Optimization Algorithm	Mathematical Update Rule	Key Advantages in Deep Learning
Stochastic Gradient Descent (SGD)	$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_i(\theta_t)$	Fast per-iteration computation; noisy gradients escape shallow saddle points.
Momentum SGD	$v_{t+1} = \gamma v_t + \eta \nabla_\theta \mathcal{L}(\theta_t), \quad \theta_{t+1} = \theta_t - v_{t+1}$	Dampens oscillations across high-curvature ravines; accelerates progress along flat valleys.
Adam (Adaptive Moment Estimation)	$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ $\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \ \hat{v}_t = \frac{v_t}{1-\beta_2^t} \implies \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t$	Combines momentum with adaptive per-parameter learning rates; the universal standard for Transformers and Deep Neural Networks.

🏛️ Step-by-Step Solved Problem: Support Vector Machine Dual Formulation & Support Vector Identification

Given 3 training data points in 2D space: $\mathbf{x}_1 = (1,1)^T, y_1 = -1; \mathbf{x}_2 = (2,0)^T, y_2 = -1; \mathbf{x}_3 = (2,3)^T, y_3 = +1$.

1. SVM Dual Formulation:

$$\max_{\alpha} \sum_{i=1}^{3} \alpha_i - \frac{1}{2} \sum_{i=1}^{3} \sum_{j=1}^{3} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i^T \mathbf{x}_j) \quad \text{subject to } \sum \alpha_i y_i = 0, \ \alpha_i \geq 0$$

2. Optimal Solution: $\alpha_1 = 0, \ \alpha_2 = 0.5, \ \alpha_3 = 0.5$.

$$\mathbf{w} = \sum_{i=1}^{3} \alpha_i y_i \mathbf{x}_i = 0.5(-1)(2,0)^T + 0.5(+1)(2,3)^T = (-1,0)^T + (1,1.5)^T = (0,1.5)^T$$

3. Hyperplane Bias b: For support vector $\mathbf{x}_3$: $y_3(\mathbf{w}^T \mathbf{x}_3 + b) = 1 \implies 1(0(2) + 1.5(3) + b) = 1 \implies 4.5 + b = 1 \implies \mathbf{b = -3.5}$.

$$\text{Optimal Decision Boundary: } 1.5x_2 - 3.5 = 0 \iff x_2 = 2.333$$

Topic 38.7: Advanced Deep Reinforcement Learning (DQN & Policy Gradients)

Deep RL Algorithm	Core Loss Function & Update Mechanism	Key Innovations
Deep Q-Networks (DQN)	$\mathcal{L}(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right]$	Experience Replay Buffer (breaks temporal correlation), Target Network θ^- (stabilizes targets).
REINFORCE (Policy Gradient)	$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{T} \nabla_\theta \ln \pi_\theta(a_t \mid s_t) G_t\right]$	Direct optimization of parameterized policy $\pi_\theta(a \mid s)$ without action-value discretization.
Actor-Critic (A2C / PPO)	Actor updates $\pi_\theta(a \mid s)$ using Advantage $A(s,a) = Q(s,a) - V(s)$; Critic updates value function $V_\phi(s)$ via MSE loss.	Significantly reduces gradient variance while retaining high sample efficiency.

🏛️ Step-by-Step Solved Problem: Policy Gradient (REINFORCE) Parameter Update Calculation

A policy network outputs action probabilities $\pi_\theta(a_1 \mid s) = 0.70, \pi_\theta(a_2 \mid s) = 0.30$ using Softmax over linear logits $z_1 = \theta_1 x, z_2 = \theta_2 x$ with $x = 1.0$. The agent executes action a_1 and receives return $G_t = +5.0$. Learning rate $\eta = 0.1$.

1. Log-Likelihood Gradient for Softmax:

$$\nabla_{\theta_1} \ln \pi_\theta(a_1 \mid s) = x(1 - \pi_\theta(a_1 \mid s)) = 1.0(1 - 0.70) = +\mathbf{0.30}$$

$$\nabla_{\theta_2} \ln \pi_\theta(a_1 \mid s) = -x \cdot \pi_\theta(a_2 \mid s) = -1.0(0.30) = -\mathbf{0.30}$$

2. Policy Parameter Updates:

$$\Delta\theta_1 = \eta \cdot \nabla_{\theta_1} \ln \pi(a_1 \mid s) \cdot G_t = 0.1 \times 0.30 \times 5.0 = +\mathbf{0.15}$$

$$\Delta\theta_2 = \eta \cdot \nabla_{\theta_2} \ln \pi(a_1 \mid s) \cdot G_t = 0.1 \times (-0.30) \times 5.0 = -\mathbf{0.15}$$

Result: θ_1 is boosted by $+0.15$, increasing the probability of choosing successful action a_1 in state s on future episodes!

Topic 38.8: Complete Step-by-Step Solved Problem Bank (Part III)

🏛️ Solved Numerical 4: C4.5 Decision Tree Gain Ratio Calculation

A dataset of 20 samples has target entropy $H(S) = 0.95$. An attribute **Outlook** with values $\{\text{Sunny: } 8, \text{Overcast: } 6, \text{Rain: } 6\}$ achieves Information Gain $\text{Gain}(S, \text{Outlook}) = 0.246$ bits.

1. Calculate Split Information:

$$\text{SplitInfo}(\mathbf{S}, \text{Outlook}) = -\sum_{\mathbf{i=1}}^{\mathbf{3}} \frac{|\mathbf{S_i}|}{|\mathbf{S}|} \log_{\mathbf{2}} \left(\frac{|\mathbf{S_i}|}{|\mathbf{S}|} \right) = -\left[\frac{\mathbf{8}}{\mathbf{20}} \log_{\mathbf{2}} \left(\frac{\mathbf{8}}{\mathbf{20}} \right) + \frac{\mathbf{6}}{\mathbf{20}} \log_{\mathbf{2}} \left(\frac{\mathbf{6}}{\mathbf{20}} \right) + \frac{\mathbf{6}}{\mathbf{20}} \log_{\mathbf{2}} \left(\frac{\mathbf{6}}{\mathbf{20}} \right) \right]$$

$$= -[0.40(-1.3219) + 0.30(-1.737) + 0.30(-1.737)] = -[-0.5288 - 0.5211 - 0.5211] = \mathbf{1.571} \text{ bits}$$

2. Calculate Gain Ratio:

$$\text{GainRatio}(\mathbf{S}, \text{Outlook}) = \frac{\text{Gain}(\mathbf{S}, \text{Outlook})}{\text{SplitInfo}(\mathbf{S}, \text{Outlook})} = \frac{\mathbf{0.246}}{\mathbf{1.571}} = \mathbf{0.1566} = \mathbf{15.66\%}$$

Significance: Gain Ratio penalizes broad multi-valued attributes, preventing overfitting on high-cardinality features (like Customer ID)!

🏛️ Step-by-Step Solved Problem: K-Means Clustering Centroid Convergence Trace

Cluster 4 points $P_1(1,1), P_2(2,1), P_3(4,3), P_4(5,4)$ into $K = 2$ clusters with initial seeds $C_1 = (1,1)$ and $C_2 = (2,1)$:

1. **Distance Computation (Iteration 1):**
   - $P_1(1,1) : d(P_1, C_1) = 0, d(P_1, C_2) = 1 \implies$ Cluster 1.
   - $P_2(2,1) : d(P_2, C_1) = 1, d(P_2, C_2) = 0 \implies$ Cluster 2.
   - $P_3(4,3) : d(P_3, C_1) = \sqrt{3^2 + 2^2} = \sqrt{13} \approx 3.61;\ d(P_3, C_2) = \sqrt{2^2 + 2^2} = \sqrt{8} \approx 2.83 \implies$ Cluster 2.
   - $P_4(5,4) : d(P_4, C_1) = \sqrt{4^2 + 3^2} = 5.0;\ d(P_4, C_2) = \sqrt{3^2 + 3^2} = \sqrt{18} \approx 4.24 \implies$ Cluster 2.
2. **Update Centroids:**

$$C_1^{\text{new}} = (1,1)$$

$$C_2^{\text{new}} = \left(\frac{2+4+5}{3}, \frac{1+3+4}{3} \right) = \left(\frac{11}{3}, \frac{8}{3} \right) = (\mathbf{3.67}, \mathbf{2.67})$$

3. **Iteration 2:** Assignments remain unchanged. **Converged!**

Q13. Explain AdaBoost Algorithm and Weighted Training Sample Updates. (8 Marks)

AdaBoost trains a sequence of weak learners $h_t(x)$:
1. Initialize sample weights $w_{1,i} = \frac{1}{N}$.
2. In iteration t, compute weighted error $\epsilon_t = \sum_{i:y_i \neq h_t(x_i)} w_{t,i}$.
3. Compute learner weight $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$.
4. Update instance weights: $w_{t+1,i} = \frac{w_{t,i} \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ (misclassified samples get amplified weights!).
5. Final ensemble: $H(x) = \text{sign} \left(\sum_{t=1}^{T} \alpha_t h_t(x) \right)$.

Topic 38.9: Advanced Statistical Machine Learning & KKT Support Vector Bounds

The Karush-Kuhn-Tucker (KKT) Optimality Conditions for Soft-Margin SVM: For primal objective $\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i$ with constraints $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$: 1. **Stationarity:** $\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \mathbf{x}_i, \quad \sum_{i=1}^{N} \alpha_i y_i = 0, \quad C - \alpha_i - \mu_i = 0.$
2. **Primal Feasibility:** $y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 + \xi_i \geq 0, \quad \xi_i \geq 0.$
3. **Dual Feasibility:** $0 \leq \alpha_i \leq C, \quad \mu_i \geq 0.$
4. **Complementary Slackness:** $\alpha_i[y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 + \xi_i] = 0 \quad \text{and} \quad \mu_i\xi_i = (C - \alpha_i)\xi_i = 0.$

🏛️ Step-by-Step Solved Problem: Support Vector Classification Case Analysis

From KKT Complementary Slackness, data points fall into three mathematically distinct regimes based on α_i:

- **Case 1 ($\alpha_i = 0$):** Point is strictly on the correct side of the margin ($y_i(\mathbf{w}^T\mathbf{x}_i + b) > 1, \xi_i = 0$). It has zero influence on the decision boundary!
- **Case 2 ($0 < \alpha_i < C$):** Point is a **Free Support Vector** lying exactly on the margin ($y_i(\mathbf{w}^T\mathbf{x}_i + b) = 1, \xi_i = 0$). These points determine the exact location of the hyperplane bias b!
- **Case 3 ($\alpha_i = C$):** Point is a **Bounded Support Vector** violating the margin ($\xi_i > 0$). It lies inside the margin band or is misclassified.

🏛️ Step-by-Step Solved Problem: Deep Multilayer Perceptron Matrix Form

Consider a 3-layer neural network with layer dimensions $d_0 \to d_1 \to d_2 \to d_3$:

$$\mathbf{a}^{(0)} = \mathbf{x} \in \mathbb{R}^{\mathbf{d_0}}$$

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{a}^{(0)} + \mathbf{b}^{(1)} \in \mathbb{R}^{\mathbf{d_1}}, \quad \mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)})$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)} \in \mathbb{R}^{\mathbf{d_2}}, \quad \mathbf{a}^{(2)} = \sigma(\mathbf{z}^{(2)})$$

$$\mathbf{z}^{(3)} = \mathbf{W}^{(3)}\mathbf{a}^{(2)} + \mathbf{b}^{(3)} \in \mathbb{R}^{\mathbf{d_3}}, \quad \hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(3)})$$

Cross-Entropy Loss with Softmax Output has the remarkably clean gradient:

$$\delta^{(3)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(3)}} = \hat{\mathbf{y}} - \mathbf{y}$$

$$\delta^{(2)} = (\mathbf{W}^{(3)})^{\mathbf{T}}\delta^{(3)} \odot \sigma'(\mathbf{z}^{(2)}) \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \delta^{(2)}(\mathbf{a}^{(1)})^{\mathbf{T}}$$

Topic 38.10: Master University Exam Problem Bank (Part IV)

🏛️ Solved Numerical 5: Neural Network Multiclass Cross-Entropy Loss Derivation

An image classifier outputs raw unnormalized logits for 3 classes: $\mathbf{z} = [2.0, 1.0, 0.1]^T$. True class label is Class 1 ($\mathbf{y} = [1, 0, 0]^T$).

1. Compute Softmax Probabilities:

$$\sum e^{z_i} = e^{2.0} + e^{1.0} + e^{0.1} = 7.389 + 2.718 + 1.105 = \mathbf{11.212}$$

$$p_1 = \frac{7.389}{11.212} = \mathbf{0.6590} \qquad p_2 = \frac{2.718}{11.212} = \mathbf{0.2424} \qquad p_3 = \frac{1.105}{11.212} = \mathbf{0.0986}$$

2. Compute Categorical Cross-Entropy Loss:

$$\mathcal{L} = -\sum_{\mathbf{i=1}}^{\mathbf{3}} \mathbf{y_i} \ln(\mathbf{p_i}) = -\mathbf{1.0} \ln(\mathbf{0.6590}) = -(\mathbf{-0.417}) = \mathbf{0.417} \text{ nats}$$

Q14. Explain Batch Normalization, Layer Normalization, and Dropout Regularization. (8 Marks)

• **Batch Normalization (Ioffe & Szegedy):** Normalizes neuron activations across mini-batch samples ($\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, y = \gamma \hat{x} + \beta$). Accelerates convergence and stabilizes internal covariate shift.
• **Layer Normalization (Ba, Kiros, Hinton):** Normalizes across all hidden features of a single sample independent of batch size; the universal standard for Transformer attention blocks.
• **Dropout (Srivastava et al.):** Randomly zeros out hidden neuron activations with probability p during training, preventing complex co-adaptations and acting as an ensemble over 2^N sub-networks.

HANDS-ON PRACTICAL LAB MASTER APPENDIX

AI Laboratory & Python Heuristic Search Master Guide

Complete Python Algorithms for A* Search, 8–Puzzle Heuristics, Alpha-Beta Game Trees & Backpropagation

Lab Experiment 1: Production-Grade A* Search on 8-Puzzle in Python

```
import heapq

class Puzzle8State:
    def __init__(self, board, parent=None, action=None, g=0):
        self.board = board
        self.parent = parent
        self.action = action
        self.g = g
        self.h = self.calculate_manhattan()
        self.f = self.g + self.h

    def calculate_manhattan(self):
        goal_pos = {1:(0,0), 2:(0,1), 3:(0,2), 4:(1,0), 5:(1,1), 6:(1,2), 7:(2,0), 8:(2,1), 0:(2,2)}
        dist = 0
        for idx, val in enumerate(self.board):
            if val != 0:
                r, c = idx // 3, idx % 3
                gr, gc = goal_pos[val]
                dist += abs(r - gr) + abs(c - gc)
        return dist

    def get_neighbors(self):
        neighbors = []
        idx = self.board.index(0)
        r, c = idx // 3, idx % 3
        moves = [(-1, 0, 'UP'), (1, 0, 'DOWN'), (0, -1, 'LEFT'), (0, 1, 'RIGHT')]
        for dr, dc, act in moves:
            nr, nc = r + dr, c + dc
            if 0 <= nr < 3 and 0 <= nc < 3:
                n_idx = nr * 3 + nc
                b_list = list(self.board)
                b_list[idx], b_list[n_idx] = b_list[n_idx], b_list[idx]
                neighbors.append(Puzzle8State(tuple(b_list), parent=self, action=act, g=self.g + 1))
        return neighbors

    def __lt__(self, other):
        return self.f < other.f

def solve_8_puzzle_astar(start_board):
    start_state = Puzzle8State(start_board)
    frontier = []
    heapq.heappush(frontier, start_state)
    explored = set()

    while frontier:
        current = heapq.heappop(frontier)
        if current.h == 0:
            path = []
            curr = current
            while curr.parent:
                path.append(curr.action)
                curr = curr.parent
            return path[::-1], current.g

        explored.add(current.board)
        for neighbor in current.get_neighbors():
            if neighbor.board not in explored:
                heapq.heappush(frontier, neighbor)
    return None, -1

# Example Run
initial = (1, 2, 3, 0, 4, 6, 7, 5, 8)
path, cost = solve_8_puzzle_astar(initial)
print(f"Optimal Moves ({cost} steps): {path}")
```

COMPREHENSIVE REVISION APPENDIX

10-Page Master Quick Revision Guide

Formulas, Algorithm Checklists & Solved Exam Cards

Page 1: AI Master Mental Model & Environmental Taxonomy

Core Paradigm

Rational Agent: Selects action $a \in \mathcal{A}$ maximizing expected performance measure P given historical percept sequence $\mathcal{P}^*$.

Environment	Observability	Determinism	Episodic/Seq	Static/Dyn	Discrete/Cont	Agents
Chess	Fully	Deterministic	Sequential	Static	Discrete	Multi (Adversarial)
Poker	Partially	Stochastic	Sequential	Static	Discrete	Multi (Adversarial)
Self-Driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Medical Diagn.	Partially	Stochastic	Sequential	Dynamic	Continuous	Single

Page 2: Uninformed vs. Informed Search Complexity Matrix

Algorithm	Data Structure	Time Complexity	Space Complexity	Complete?	Optimal?
BFS	FIFO Queue	$O(b^d)$	$O(b^d)$	Yes	Yes (uniform costs)
DFS	LIFO Stack	$O(b^m)$	$O(b \cdot m)$	No (infinite trees)	No
UCS	Priority Queue	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	Yes	**Yes (Cost optimal)**
IDDFS	LIFO Stack	$O(b^d)$	$\mathbf{O(b \cdot d)}$	Yes	Yes (uniform costs)
A^* **Search**	Priority Queue	$O(b^d)$	$O(b^d)$	Yes	**Yes (if h admissible)**

Page 3: A^* Search Admissibility & Consistency Theorems

$$\text{Admissibility: } 0 \leq h(n) \leq h^*(n) \implies A^* \text{ Tree Search is Optimal}$$

$$\text{Consistency: } h(n) \leq c(n, a, n') + h(n') \implies A^* \text{ Graph Search is Optimal (No node re-expansion!)}$$

Page 4: Adversarial Minimax & Alpha-Beta Pruning Rules

$$\alpha \leftarrow \max(\alpha, \text{child\_val}) \quad (\text{Initialized to } -\infty)$$

$$\beta \leftarrow \min(\beta, \text{child\_val}) \quad (\text{Initialized to } +\infty)$$

$$\text{PRUNE SUBTREE IF } \alpha \geq \beta$$

Page 5: Propositional & First-Order Logic (FOL) Transformation Rules

$$\alpha \to \beta \equiv \neg\alpha \lor \beta \qquad \alpha \leftrightarrow \beta \equiv (\neg\alpha \lor \beta) \land (\neg\beta \lor \alpha)$$

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x) \qquad \neg(\exists x P(x)) \equiv \forall x \neg P(x)$$

$$\text{Resolution: } \frac{A \lor B, \quad \neg B \lor C}{A \lor C}$$

Page 6: Classical Planning STRIPS & Graphplan Mutexes

3 Mutex Conditions

Inconsistent Effects (Add vs Delete), Interference (Delete vs Precond), Competing Needs (Mutex Preconditions).

Page 7: Probability, Bayes' Rule & Bayesian Networks

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)} \qquad P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{Parents}(X_i))$$

Page 8: Decision Tree Induction (ID3 Entropy & Gain)

$$H(S) = -\sum p_i \log_2(p_i) \qquad \text{Gain}(S, A) = H(S) - \sum \frac{|S_v|}{|S|} H(S_v)$$

Page 9: Multi-Layer Perceptrons & Backpropagation Gradient

$$\delta_k = (y_k - \hat{y}_k)\sigma'(z_k) \qquad \delta_j = \sigma'(z_j)\sum_k \delta_k w_{jk} \qquad \Delta w = \eta \delta a_{\text{in}}$$

Page 10: AI Final High-Yield Exam Checklist & Formulas

10 Instant Revision Points

1. A^* with $h = 0$ becomes Uniform Cost Search.
2. A^* with $g = 0$ becomes Greedy Best-First Search.
3. Iterative Deepening DFS uses $O(bd)$ memory and visits nodes at depth d only once.
4. Alpha-Beta does not affect final minimax value at root.
5. Skolemization replaces $\exists x$ with constant or Skolem function $f(y_1 \dots y_k)$.
6. Unification requires variables to be standardized apart.
7. d-separation isolates nodes given evidence variables.
8. Backpropagation computes exact gradient via dynamic programming chain rule.
9. ReLU eliminates vanishing gradient for positive activations.
10. Softmax produces valid categorical probability distribution.